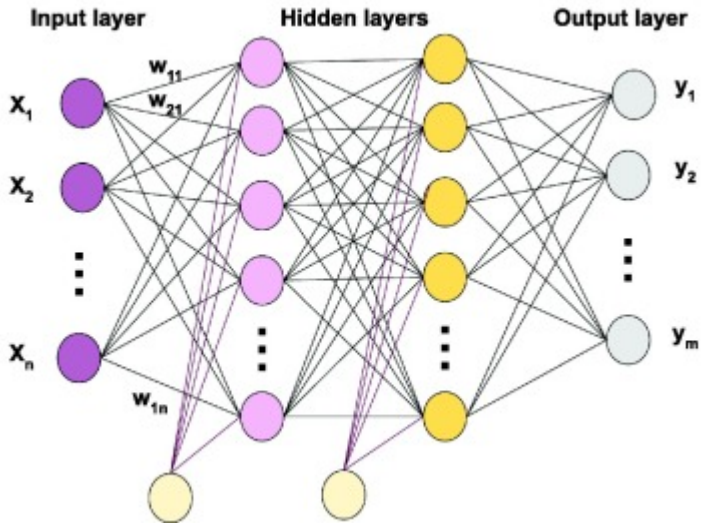
Practical No - 1

Aim: Implement multilayer perceptron algorithm for MNIST Hand written Digit Classification.
Theory: To understand this experiment, we look at the Multi-Layer Perceptron (MLP), which is a specific type of Feed-Forward Neural Network. It is designed to mimic how biological neurons in the brain process information to make decisions.

A. The Artificial Neuron

Imagine a neuron as a tiny decision-making unit. It receives information from several sources, assigns a certain level of "importance" (called weight) to each source, and adds a baseline value (called bias).

Once it combines all this information, it passes the result through an Activation Function. This function decides whether the neuron should "fire" (pass a signal forward) or stay inactive. In our network, thousands of these neurons work together to recognize patterns in the digit images.



B. Network Architecture

Our MLP is built like a sandwich with three specific types of layers:

1. Input Layer: This is the entry point. Since the computer sees images as a grid of pixels, we must first "flatten" the 2D image into a single long line of numbers. Each number represents the brightness of one pixel.
2. Hidden Layers: These are the layers in the middle where the actual learning happens. Our code uses "Dense" layers, meaning every neuron in one layer is connected to every neuron in the next. These layers look for features like edges, curves, and loops.

3. Output Layer: This is the final layer. Since we are classifying digits from 0 to 9, this layer has exactly 10 neurons. The neuron with the highest value tells us the network's final prediction.

C. Key Concepts in my Code

- ReLU (Rectified Linear Unit): This is an activation function used in the hidden layers. It acts like a gatekeeper: if a value is positive, it lets it through unchanged; if it is negative, it turns it to zero. This simple rule allows the network to learn non-linear, complex patterns.
- Softmax: This activation function is used only in the output layer. It takes the raw scores from the network and converts them into probabilities (percentages). This allows the model to say, "I am 98% sure this is a 7."
- Dropout: This is a regularization technique. During training, it randomly "turns off" a percentage of neurons (20% in our code). This forces the network not to rely too heavily on any single neuron, making the final model smarter and more robust (preventing overfitting).
- Backpropagation & Optimizer (Adam): The network starts by guessing randomly. After each guess, it checks how wrong it was (the Loss). The Optimizer then goes backward through the network, tweaking the weights slightly to reduce the error for the next time. "Adam" is a smart optimizer that adjusts these tweaks efficiently.

Algorithm:

Step 1: Initialization Import the necessary libraries (TensorFlow, Keras, NumPy, Matplotlib) and set the random seeds to ensure the experiment results are consistent and reproducible.

Step 2: Data Loading and Preprocessing Load the MNIST dataset and split it into training and testing sets. Normalize the pixel intensity values by dividing by 255, converting them from integers to floating-point numbers between 0 and 1.

Step 3: Model Architecture Design Construct the Sequential model with the following layers:

- Add a Flatten layer to convert the 28x28 image grid into a 1D vector.
- Add a Dense hidden layer with 256 neurons and ReLU activation.
- Add a Dropout layer (0.2) to prevent overfitting.
- Add a second Dense hidden layer with 128 neurons and ReLU activation, followed by Dropout.
- Add a third Dense hidden layer with 64 neurons and ReLU activation, followed by Dropout.
- Add a final Output Dense layer with 10 neurons (representing digits 0-9) using Softmax activation.

Step 4: Compilation Configure the model training process using the 'Adam' optimizer and 'Sparse Categorical Crossentropy' as the loss function. Set 'Accuracy' as the metric to monitor.

Step 5: Training (Fitting) Train the model using the training data for 30 epochs with a batch size of 128. Implement callbacks for 'Early Stopping' (to stop training if validation loss stabilizes) and 'Reduce Learning Rate' (to fine-tune weights if progress slows).

Step 6: Evaluation Run the trained model on the unseen test dataset to calculate the final Test Loss and Test Accuracy.

Step 7: Visualization Generate plots for Accuracy vs. Epochs and Loss vs. Epochs to analyze training performance. Display a sample of test images alongside their predicted labels to visually verify the model's success.

Dataset Description: The MNIST dataset is the standard benchmark for learning image classification.

- Content: It contains 70,000 grayscale images of handwritten digits (0-9).
- Split: It is divided into 60,000 images for training (teaching the model) and 10,000 for testing (checking how well it learned).
- Format: Each image is small, measuring 28 × 28 pixels.
- Preprocessing: The original pixel darkness values range from 0 to 255. In the code, we divide these by 255.0 to squash them into a range of 0 to 1. This "normalization" helps the neural network learn faster and more accurately.

Source Code:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt

tf.random.set_seed(42)
np.random.seed(42)

def main():
    (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
    x_train, x_test = x_train.astype('float32') / 255.0,
x_test.astype('float32') / 255.0
    model = keras.Sequential([
        layers.Flatten(input_shape=(28, 28)),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.2),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.2),
        layers.Dense(64, activation='relu'),
        layers.Dropout(0.2),
        layers.Dense(10, activation='softmax')
    ])
```

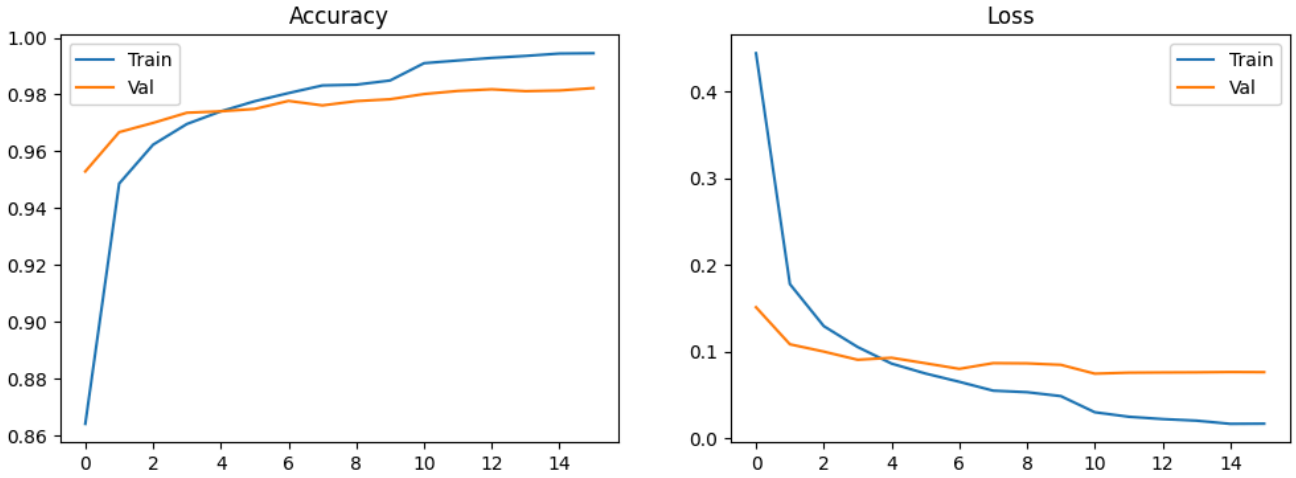
```
    model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

    callbacks = [
        keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True),
        keras.callbacks.ReduceLROnPlateau(factor=0.2, patience=3)
    ]

    history = model.fit(x_train, y_train, epochs=30, batch_size=128,
                        validation_split=0.2, callbacks=callbacks, verbose=1)

    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
    print(f"Test Loss: {test_loss:.4f}, Test Accuracy: {test_acc:.4f}")

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
    ax1.plot(history.history['accuracy'], label='Train'),
ax1.plot(history.history['val_accuracy'], label='Val')
    ax1.set_title('Accuracy'), ax1.legend()
    ax2.plot(history.history['loss'], label='Train'),
ax2.plot(history.history['val_loss'], label='Val')
    ax2.set_title('Loss'), ax2.legend()
    plt.show()

    preds = model.predict(x_test[:10])
    fig, axes = plt.subplots(2, 5, figsize=(10, 5))
    for i, ax in enumerate(axes.flat):
        ax.imshow(x_test[i], cmap='gray')
        ax.axis('off')
        color = 'green' if np.argmax(preds[i]) == y_test[i] else 'red'
        ax.set_title(f"P:{np.argmax(preds[i])} | T:{y_test[i]}", color=color)
    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    main()
```

Output:

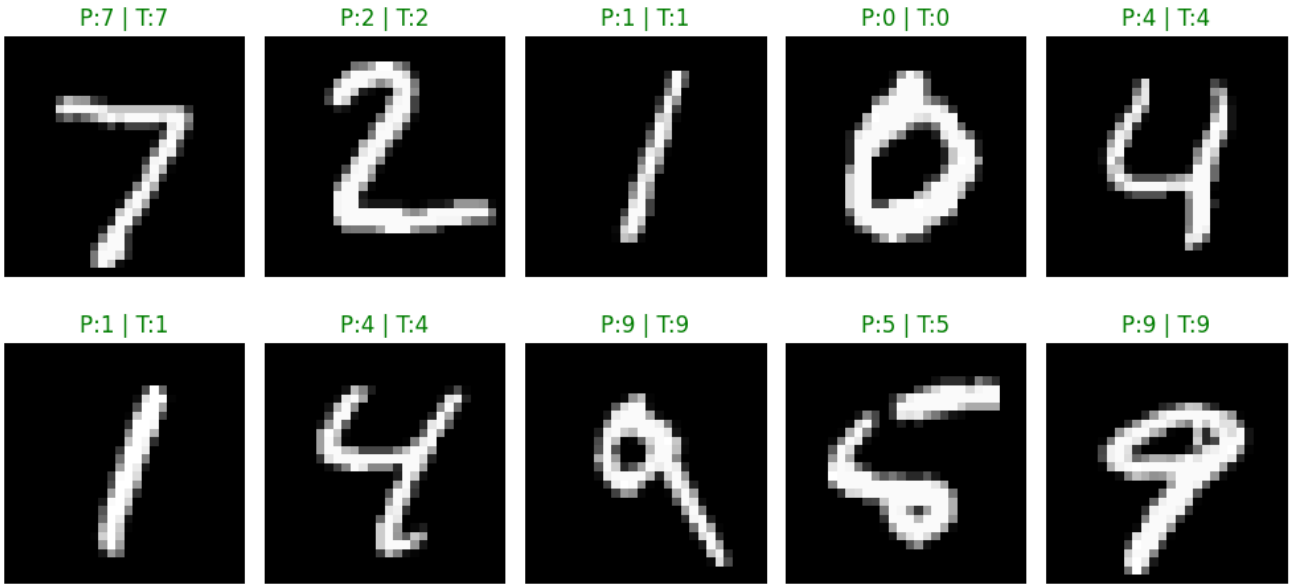
```
Epoch 1/30
375/375 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.7498 - loss: 0.7781 -
val_accuracy: 0.9529 - val_loss: 0.1513 - learning_rate: 0.0010
```

Epoch 2/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **4s** 9ms/step - accuracy: 0.9430 - loss: 0.1936 - val_accuracy: 0.9668 - val_loss: 0.1086 - learning_rate: 0.0010
Epoch 3/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **5s** 13ms/step - accuracy: 0.9610 - loss: 0.1363 - val_accuracy: 0.9700 - val_loss: 0.1001 - learning_rate: 0.0010
Epoch 4/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **4s** 9ms/step - accuracy: 0.9675 - loss: 0.1117 - val_accuracy: 0.9736 - val_loss: 0.0907 - learning_rate: 0.0010
Epoch 5/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **4s** 9ms/step - accuracy: 0.9736 - loss: 0.0878 - val_accuracy: 0.9741 - val_loss: 0.0930 - learning_rate: 0.0010
Epoch 6/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **5s** 13ms/step - accuracy: 0.9761 - loss: 0.0773 - val_accuracy: 0.9749 - val_loss: 0.0867 - learning_rate: 0.0010
Epoch 7/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **4s** 9ms/step - accuracy: 0.9805 - loss: 0.0653 - val_accuracy: 0.9778 - val_loss: 0.0802 - learning_rate: 0.0010
Epoch 8/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **4s** 9ms/step - accuracy: 0.9833 - loss: 0.0544 - val_accuracy: 0.9762 - val_loss: 0.0868 - learning_rate: 0.0010
Epoch 9/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **4s** 11ms/step - accuracy: 0.9838 - loss: 0.0527 - val_accuracy: 0.9777 - val_loss: 0.0865 - learning_rate: 0.0010
Epoch 10/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **4s** 12ms/step - accuracy: 0.9846 - loss: 0.0513 - val_accuracy: 0.9783 - val_loss: 0.0848 - learning_rate: 0.0010
Epoch 11/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **4s** 9ms/step - accuracy: 0.9898 - loss: 0.0342 - val_accuracy: 0.9802 - val_loss: 0.0746 - learning_rate: 2.0000e-04
Epoch 12/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **4s** 10ms/step - accuracy: 0.9922 - loss: 0.0244 - val_accuracy: 0.9812 - val_loss: 0.0758 - learning_rate: 2.0000e-04
Epoch 13/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **5s** 9ms/step - accuracy: 0.9926 - loss: 0.0221 - val_accuracy: 0.9818 - val_loss: 0.0760 - learning_rate: 2.0000e-04
Epoch 14/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **4s** 10ms/step - accuracy: 0.9938 - loss: 0.0199 - val_accuracy: 0.9812 - val_loss: 0.0762 - learning_rate: 2.0000e-04
Epoch 15/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **6s** 13ms/step - accuracy: 0.9942 - loss: 0.0182 - val_accuracy: 0.9814 - val_loss: 0.0766 - learning_rate: 4.0000e-05
Epoch 16/30
375/375 ━━━━━━━━━━━━━━━━━━━━ **4s** 9ms/step - accuracy: 0.9944 - loss: 0.0177 - val_accuracy: 0.9822 - val_loss: 0.0764 - learning_rate: 4.0000e-05

Test Loss: 0.0657, Test Accuracy: 0.9823



1/1 ——————————————————— **0s** 77ms/step



Learning Outcome:

Practical No - 2

Aim: Design a neural network for classifying movie reviews (Binary Classification) using IMDBdataset.
Theory:

1. Theoretical Background

A. Introduction to Sentiment Analysis Sentiment Analysis is a subfield of Natural Language Processing (NLP) that involves determining the emotional tone behind a body of text. In this experiment, we perform Binary Classification, where the goal is to categorize movie reviews into one of two distinct classes: Positive or Negative. Unlike image data, text is sequential and unstructured, requiring specific mathematical transformations before a neural network can process it.

B. The IMDB Dataset The IMDB dataset is a standard benchmark for binary sentiment classification. It contains 50,000 highly polarized movie reviews. The data is split into 25,000 reviews for training and 25,000 for testing. Each review is labeled as either 0 (negative) or 1 (positive). In Keras, these reviews are pre-encoded as sequences of integers, where each integer represents a specific word in a dictionary.

C. Word Embeddings Computers cannot understand raw words. An **Embedding Layer** is used to map each word index to a high-dimensional vector.

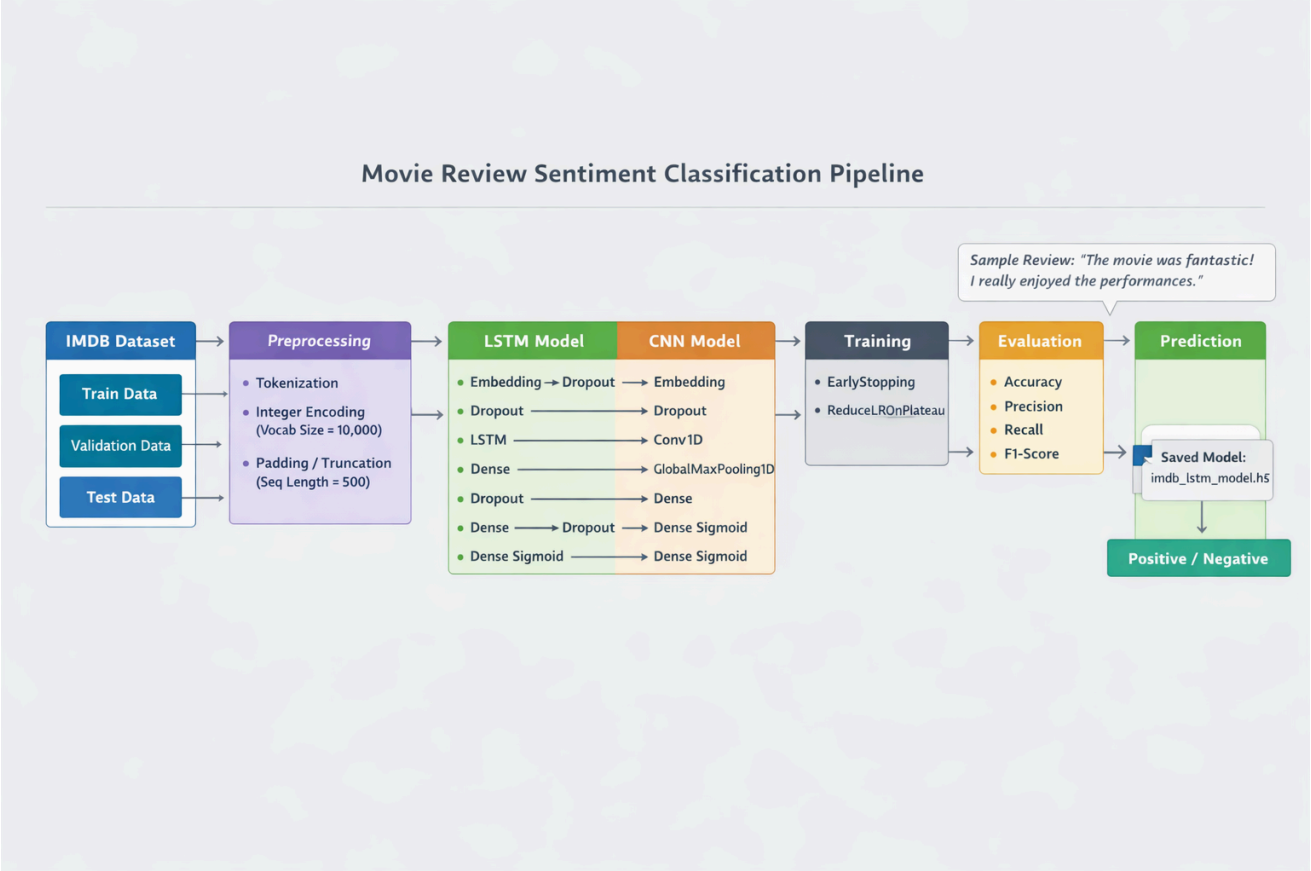
Words with similar meanings (e.g., "excellent" and "fantastic") are moved closer together in this vector space during training. This allows the model to understand the semantic relationship between words rather than just treating them as isolated IDs.

D. Neural Network Architectures for NLP This experiment explores three powerful architectures for processing text:

1. **Long Short-Term Memory (LSTM):** A type of Recurrent Neural Network (RNN) designed to remember long-term dependencies. Text is sequential; the meaning of a word often depends on words that came before it. LSTMs use "gates" to decide which information to keep or discard, making them excellent for understanding the context of a full sentence.
2. **Convolutional Neural Networks (CNN):** While usually used for images, 1D-CNNs are effective for text. They use "filters" to slide over sequences of words to detect local patterns, such as key phrases (e.g., "not good" or "must watch").
3. **Transformers:** The modern state-of-the-art architecture. Instead of processing words one by one, Transformers use a **Multi-Head Attention** mechanism to look at all words in a sentence simultaneously and determine which words are most relevant to each other.

E. Regularization and Optimization

- **Padding:** Since movie reviews have different lengths, we use "padding" to ensure all input sequences are the same length (500 words).
- **Dropout:** To prevent the model from simply memorizing the training reviews (overfitting), we randomly deactivate neurons during training.
- **Sigmoid Activation:** Since this is a binary task, the final neuron uses a Sigmoid function to squash the output between 0 and 1, representing the probability of a "Positive" sentiment.



---

2. Algorithm

Step 1: Initialization and Data Loading

- Import TensorFlow, Keras, and visualization tools.
- Load the IMDB dataset, limiting the vocabulary to the top 10,000 most frequent words to reduce noise and computational load.
- Split the training data into a 90% training set and a 10% validation set to monitor performance during the learning process.

Step 2: Text Preprocessing

- Calculate the average length of reviews.
- Apply **Sequence Padding**: Longer reviews are truncated to 500 words, and shorter reviews are padded with zeros at the end (post-padding) to create uniform input shapes.

Step 3: Model Construction (Three Approaches) The experiment implements three distinct model creators:

- **Approach A (LSTM):** Embedding layer → Dropout → LSTM layer → Dense layer → Sigmoid output.
- **Approach B (CNN):** Embedding layer → 1D Convolutional layer → Global Max Pooling → Multiple Dense layers → Sigmoid output.
- **Approach C (Transformer):** Input layer → Embedding → Multi-Head Attention → Layer Normalization → Feed-Forward Network → Global Average Pooling → Sigmoid output.

Step 4: Compilation

- Define the **Adam Optimizer** with a specific learning rate (0.001).
- Set the loss function to **Binary Crossentropy**, which is the standard for two-class problems.
- Define evaluation metrics: Accuracy, Precision, Recall, and F1-Score to get a complete picture of model performance.

Step 5: Training and Optimization

- Fit the model using a batch size of 32.
- **Early Stopping:** Monitor validation loss and stop training if it stops improving for 3 consecutive epochs to save time and prevent overfitting.
- **Learning Rate Reduction:** Automatically lower the learning rate if the model reaches a plateau, allowing for finer weight adjustments.

Step 6: Evaluation and Comparison

- Test the final weights of each model on the 25,000 unseen test samples.
- Calculate the F1-Score (the harmonic mean of precision and recall) to evaluate how well the model handles positive and negative classes.
- Save the trained models as `.h5` files for future deployment.

Step 7: Prediction and Visualization

- Plot training vs. validation graphs for all four metrics (Accuracy, Loss, Precision, Recall).
- Use a custom prediction function to process raw strings, convert them to sequences, and output a sentiment label with a confidence score.

Dataset Description: The IMDB Dataset is a collection of 50,000 highly polarized movie reviews used for binary sentiment classification. It is evenly split into 25,000 reviews for training and 25,000 for testing, where each review is labeled as either positive (1) or negative (0).

In this experiment, the dataset is pre-processed into sequences of integers, where each number corresponds to a specific word in a dictionary of 10,000 most frequent terms. To ensure consistency for the neural network, all reviews are standardized to a fixed length of 500 words using padding and truncating.

Source Code:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt
vocab_size = 10000
max_length = 250
(x_train, y_train), (x_test, y_test) =
keras.datasets.imdb.load_data(num_words=vocab_size)
x_train = keras.preprocessing.sequence.pad_sequences(x_train,
maxlen=max_length, padding='post')
x_test = keras.preprocessing.sequence.pad_sequences(x_test,
maxlen=max_length, padding='post')
model = keras.Sequential([
    layers.Embedding(input_dim=vocab_size, output_dim=16,
input_length=max_length),
    layers.GlobalAveragePooling1D(),
    layers.Dense(16, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
model.summary()
history = model.fit(
    x_train, y_train,
    epochs=10,
```

```
    batch_size=512,
    validation_split=0.2,
    verbose=1
)
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f'Test Accuracy: {test_accuracy:.4f}')
print(f'Test Loss: {test_loss:.4f}')
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes[0, 0].plot(history.history['accuracy'], label='Training Accuracy',
marker='o')
axes[0, 0].plot(history.history['val_accuracy'], label='Validation Accuracy',
marker='s')
axes[0, 0].set_title('Model Accuracy')
axes[0, 0].set_xlabel('Epoch')
axes[0, 0].set_ylabel('Accuracy')
axes[0, 0].legend()
axes[0, 0].grid(True)
axes[0, 1].plot(history.history['loss'], label='Training Loss', marker='o')
axes[0, 1].plot(history.history['val_loss'], label='Validation Loss',
marker='s')
axes[0, 1].set_title('Model Loss')
axes[0, 1].set_xlabel('Epoch')
axes[0, 1].set_ylabel('Loss')
axes[0, 1].legend()
axes[0, 1].grid(True)
train_lengths = [len(seq) for seq in x_train]
test_lengths = [len(seq) for seq in x_test]

axes[1, 0].hist(train_lengths, bins=50, alpha=0.7, label='Training',
color='blue')
axes[1, 0].hist(test_lengths, bins=50, alpha=0.7, label='Test',
color='orange')
axes[1, 0].set_title('Distribution of Sequence Lengths')
axes[1, 0].set_xlabel('Sequence Length')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].legend()
sample_predictions = model.predict(x_test[:1000])
axes[1, 1].hist(sample_predictions, bins=30, edgecolor='black')
axes[1, 1].set_title('Distribution of Predictions on Test Set (Sample)')
```

```python
axes[1, 1].set_xlabel('Prediction Probability')
axes[1, 1].set_ylabel('Frequency')
plt.tight_layout()
plt.show()
print("\nFinal Training Metrics:")
print(f"Last Epoch Training Accuracy: {history.history['accuracy'][-1]:.4f}")
print(f"Last Epoch Validation Accuracy:
{history.history['val_accuracy'][-1]:.4f}")
print(f"Last Epoch Training Loss: {history.history['loss'][-1]:.4f}")
print(f"Last Epoch Validation Loss: {history.history['val_loss'][-1]:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test Loss: {test_loss:.4f}")
```

Output:

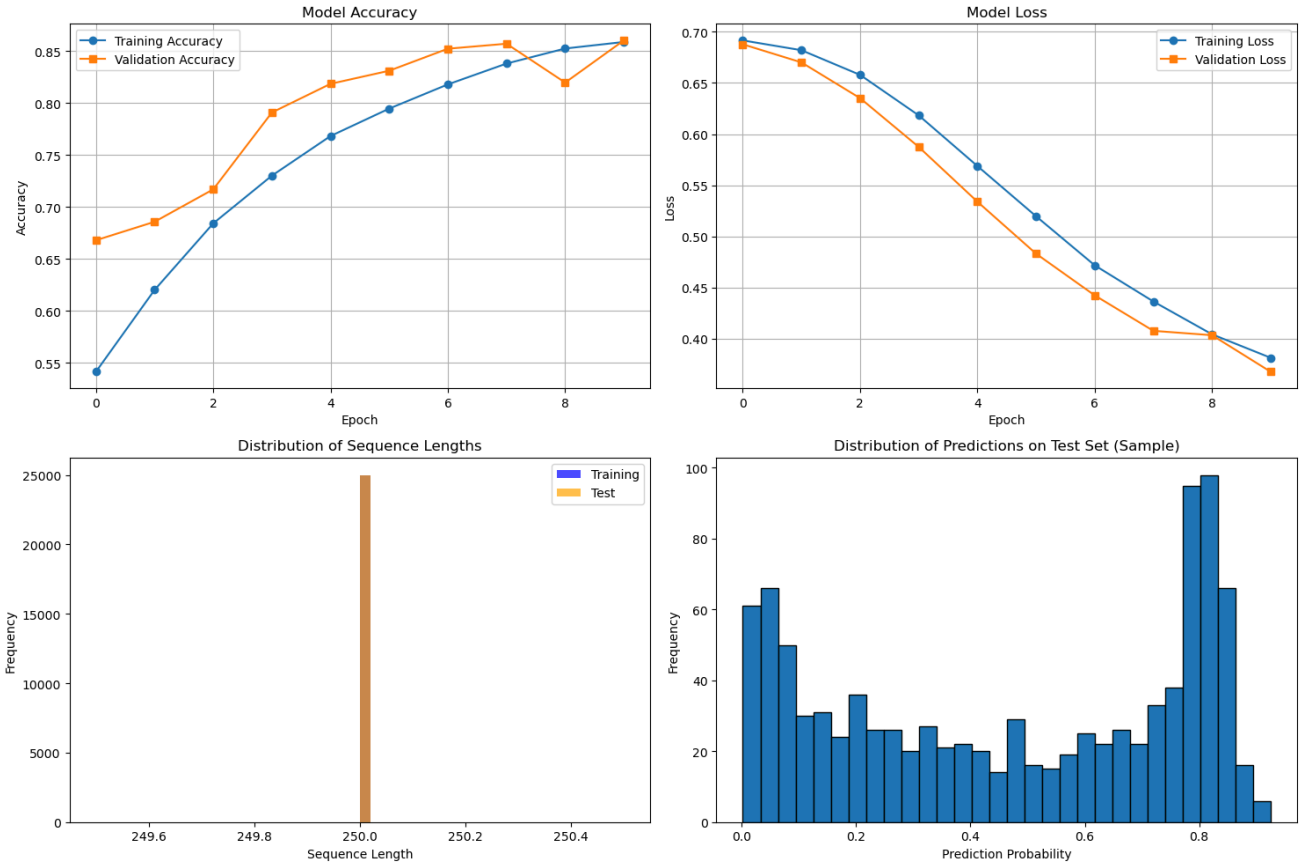
```
Model: "sequential_3"

┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ embedding_3 (Embedding)         │ ?                      │   0 (unbuilt) │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ global_average_pooling1d_1      │ ?                      │             0 │
│ (GlobalAveragePooling1D)        │                        │               │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_6 (Dense)                 │ ?                      │   0 (unbuilt) │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dropout_3 (Dropout)             │ ?                      │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_7 (Dense)                 │ ?                      │   0 (unbuilt) │
```

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.5414 -
loss: 0.6916 - val_accuracy: 0.6680 - val_loss: 0.6878
Epoch 2/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.6202 -
loss: 0.6820 - val_accuracy: 0.6858 - val_loss: 0.6701
Epoch 3/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.6842 -
loss: 0.6579 - val_accuracy: 0.7170 - val_loss: 0.6352
Epoch 4/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.7302 -
loss: 0.6183 - val_accuracy: 0.7910 - val_loss: 0.5875
Epoch 5/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7684 -
loss: 0.5688 - val_accuracy: 0.8186 - val_loss: 0.5340
Epoch 6/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.7947 -
loss: 0.5197 - val_accuracy: 0.8312 - val_loss: 0.4832
Epoch 7/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.8181 -
loss: 0.4718 - val_accuracy: 0.8524 - val_loss: 0.4424
Epoch 8/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.8381 -
loss: 0.4365 - val_accuracy: 0.8572 - val_loss: 0.4078
Epoch 9/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.8526 -
loss: 0.4044 - val_accuracy: 0.8198 - val_loss: 0.4035
Epoch 10/10
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8589 -
loss: 0.3814 - val_accuracy: 0.8604 - val_loss: 0.3678
Test Accuracy: 0.8599
Test Loss: 0.3706
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
```




```
Final Training Metrics:
Last Epoch Training Accuracy: 0.8589
Last Epoch Validation Accuracy: 0.8604
Last Epoch Training Loss: 0.3814
Last Epoch Validation Loss: 0.3678
Test Accuracy: 0.8599
     Test Loss: 0.3706
```

Learning Outcome:

Practical No - 3

Aim: Design a neural Network for classifying news wires (Multi class classification) using Reutersdataset.
Theory:

A. Overview of Multi-Class Classification In my previous experiments, I explored binary classification (two possible outcomes). However, real-world data often requires categorizing information into several distinct groups. This is known as Multi-Class Classification. In this experiment, I am tasked with classifying news articles into 46 different topics, which significantly increases the complexity of the decision-making process for the neural network.

B. The Artificial Neural Network for Categorical Data To handle 46 classes, I designed a Deep Neural Network with specific mathematical properties:

- **The Information Bottleneck Principle:** In a multi-class problem, the hidden layers must be large enough to pass complex information to the output. If I used a hidden layer with only 4 neurons to predict 46 classes, the network would face a "bottleneck," losing vital information during compression. I have used layers with 64 neurons to ensure the model has sufficient "representational space" to distinguish between subtle topics like "grain" versus "wheat."
- **Activation Functions:** I utilized the Rectified Linear Unit (ReLU) for hidden layers to allow the model to learn non-linear patterns. For the final layer, I used the **Softmax** function. Unlike Sigmoid, which is used for binary choices, Softmax turns the output scores into a probability distribution that sums exactly to 1.0. This allows me to see the model's confidence level for every single one of the 46 categories.

C. Categorical Crossentropy Loss The loss function is the heart of the learning process. For this multi-class task, I employed **Categorical Crossentropy**. This measures the "distance" between two probability distributions: the one predicted by my model and the true distribution of the labels. By minimizing this distance, the model learns to push the probability of the correct class toward 1 and all others toward 0.

D. Vectorization and Feature Engineering Text cannot be fed directly into a Dense layer. I used **One-Hot Encoding** (or Multi-hot encoding) to transform the lists of word indices into vectors of 0s and 1s. For each article, I created a vector of 10,000 dimensions (representing my vocabulary). If the 5th word in my dictionary appears in the article, the 5th index of the vector becomes a 1. This provides a consistent numerical format for the network.

Algorithm:

Step 1: Environment Setup and Library Imports I began by importing TensorFlow and Keras for model building, NumPy for numerical operations, and Matplotlib/Seaborn for visual analysis of the results.

Step 2: Data Acquisition and Restriction I loaded the Reuters dataset while applying a word limit of 10,000. This ensures that only the most relevant words are considered, preventing the model from becoming overly complex and slow.

Step 3: Data Preprocessing (Vectorization) I implemented a custom function to vectorize the sequences. This transformed the lists of integers into a 10,000-dimensional matrix of 0s and 1s. I also transformed the integer labels into categorical matrices using the `to_categorical` utility.

Step 4: Designing the Model Architecture I constructed a Sequential model with the following structure:

- **First Dense Layer:** 64 units with ReLU activation to process the 10,000 input features.
- **Second Dense Layer:** 64 units with ReLU activation to refine the feature extraction.
- **Output Layer:** 46 units with Softmax activation to generate a probability for each news category.

Step 5: Compilation I compiled the model using the **RMSprop** optimizer, which is efficient for deep learning tasks. I chose **Categorical Crossentropy** as my loss function and set **Accuracy** as the primary metric to track.

Step 6: Training with Validation I trained the model for 9 epochs with a batch size of 512. I reserved 20% of the training data as a validation set. This allowed me to monitor if the model was truly learning general patterns or simply memorizing the training data.

Step 7: Performance Visualization I generated line plots to compare training loss against validation loss and training accuracy against validation accuracy. This visual step is crucial for identifying the "sweet spot" before the model begins to overfit.

Step 8: Final Evaluation and Confusion Matrix I evaluated the final model on the unseen test dataset to determine its real-world performance. Finally, I generated a **Confusion Matrix** heatmap to see which specific categories were being confused with one another, providing a detailed map of the model's strengths and weaknesses.

Dataset Description:

A. Content and Scale The Reuters dataset is a widely used benchmark for text classification. It consists of a set of short newswires and their topics, published by Reuters in 1987. There are 46 different topics; some topics are more represented than others (like "earn" and "acq"), but each story has at least one topic assigned to it.

- **Total Samples:** The version I used contains 11,228 newswires.
- **Training/Testing Split:** I split the data into 8,982 training samples and 2,246 test samples.
- **Vocabulary:** I restricted the data to the 10,000 most frequently occurring words to ensure the model focuses on the most meaningful language patterns while ignoring rare noise.

B. Data Format Each sample is a list of integers, where each integer represents a specific word. For example, the number 1 might represent "the," and 14 might represent "reuters." This pre-encoding saves significant time in preprocessing and allows me to focus directly on the architecture of the neural network.

C. Labeling The labels are integers between 0 and 45. To prepare these for the model, I converted them into categorical "one-hot" vectors. For example, a label of 3 becomes a vector of forty-six elements where the index 3 is a '1' and all other indices are '0'.

Source Code:

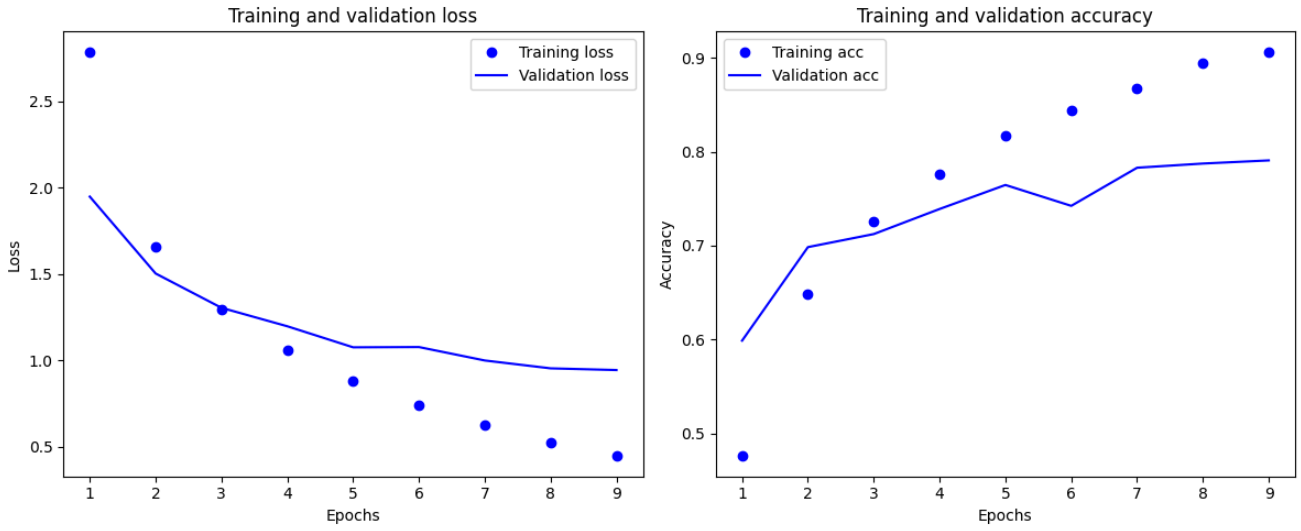
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras.datasets import reuters
from tensorflow.keras.utils import to_categorical
from tensorflow.keras import models, layers
from sklearn.metrics import confusion_matrix
(train_data, train_labels), (test_data, test_labels) =
reuters.load_data(num_words=10000)
def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))
    for i, sequence in enumerate(sequences):
        results[i, sequence] = 1.
    return results
x_train = vectorize_sequences(train_data)
x_test = vectorize_sequences(test_data)
one_hot_train_labels = to_categorical(train_labels)
one_hot_test_labels = to_categorical(test_labels)
model = models.Sequential()
model.add(layers.Dense(64, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(46, activation='softmax'))
model.compile(optimizer='rmsprop', loss='categorical_crossentropy',
metrics=['accuracy'])
history = model.fit(x_train, one_hot_train_labels, epochs=9, batch_size=512,
validation_split=0.2, verbose=1)
results = model.evaluate(x_test, one_hot_test_labels, verbose=0)
print(f"Test Loss: {results[0]}, Test Accuracy: {results[1]}")
loss = history.history['loss']
val_loss = history.history['val_loss']
```

```
epochs = range(1, len(loss) + 1)
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
plt.subplot(1, 2, 2)
plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.tight_layout()
plt.show()
predictions = model.predict(x_test)
y_pred = np.argmax(predictions, axis=1)
conf_mat = confusion_matrix(test_labels, y_pred)
plt.figure(figsize=(12, 10))
sns.heatmap(conf_mat, annot=False, fmt='d', cmap='Blues')
plt.title('Confusion Matrix (46 Classes)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
```

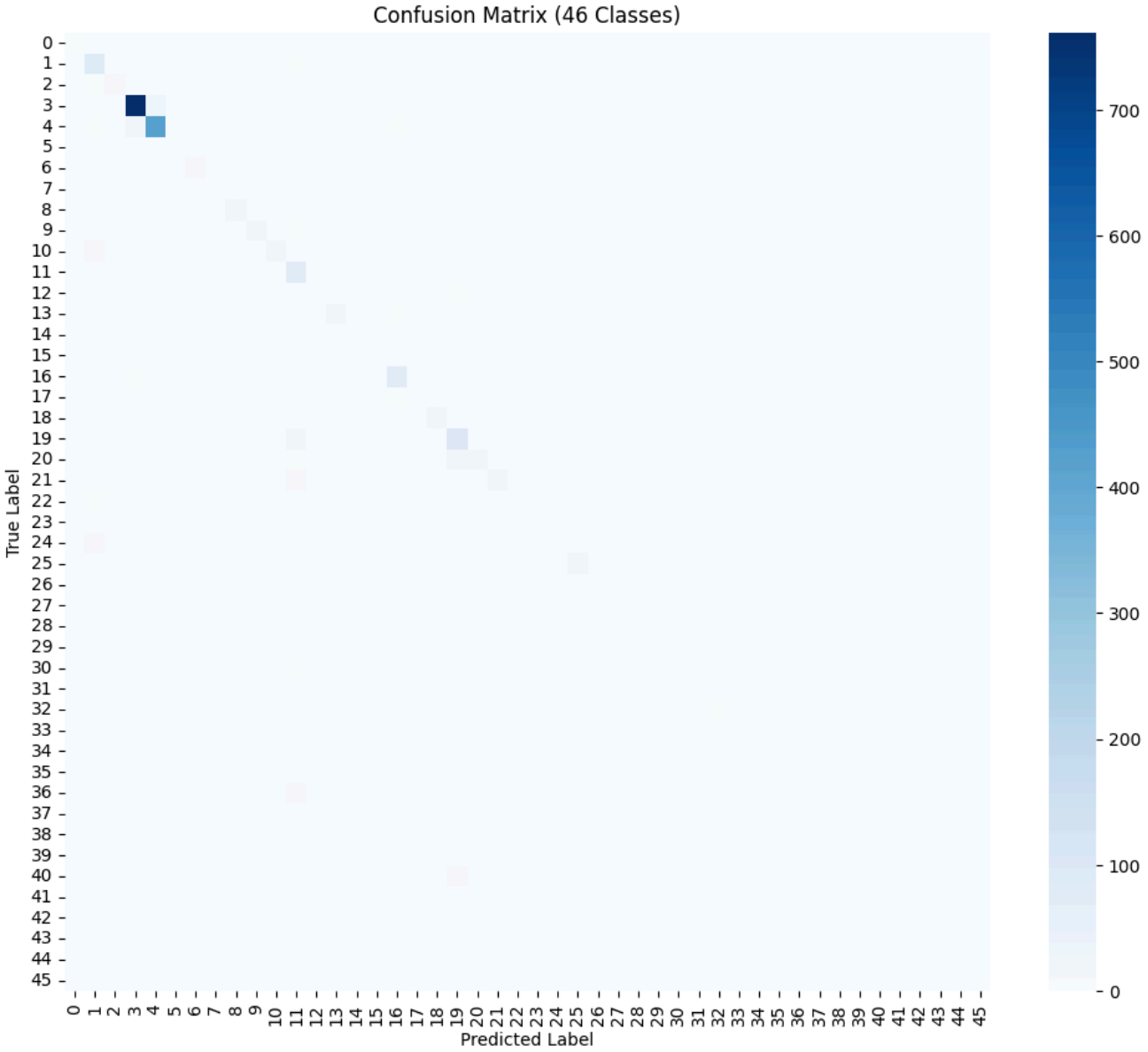
Output:

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/reuters.npz
2110848/2110848 ━━━━━━━━━━━━━━━━━━━━ **1s** 1us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/9
15/15 ━━━━━━━━━━━━━━━━━━━━ **5s** 161ms/step - accuracy: 0.3801 - loss: 3.2141 - val_accuracy: 0.5988 - val_loss: 1.9490
Epoch 2/9

15/15 **0s** 19ms/step - accuracy: 0.6342 - loss: 1.7466 - val_accuracy: 0.6984 - val_loss: 1.5039
Epoch 3/9
15/15 **0s** 19ms/step - accuracy: 0.7198 - loss: 1.3259 - val_accuracy: 0.7123 - val_loss: 1.3054
Epoch 4/9
15/15 **0s** 19ms/step - accuracy: 0.7719 - loss: 1.0730 - val_accuracy: 0.7390 - val_loss: 1.1985
Epoch 5/9
15/15 **0s** 18ms/step - accuracy: 0.8131 - loss: 0.9175 - val_accuracy: 0.7646 - val_loss: 1.0767
Epoch 6/9
15/15 **0s** 20ms/step - accuracy: 0.8468 - loss: 0.7545 - val_accuracy: 0.7423 - val_loss: 1.0782
Epoch 7/9
15/15 **0s** 19ms/step - accuracy: 0.8593 - loss: 0.6479 - val_accuracy: 0.7830 - val_loss: 1.0005
Epoch 8/9
15/15 **0s** 19ms/step - accuracy: 0.8942 - loss: 0.5286 - val_accuracy: 0.7874 - val_loss: 0.9550
Epoch 9/9
15/15 **0s** 19ms/step - accuracy: 0.9076 - loss: 0.4466 - val_accuracy: 0.7908 - val_loss: 0.9451
Test Loss: 0.9935145974159241, Test Accuracy: 0.7702582478523254



71/71 **1s** 4ms/step



Learning Outcome:

Practical No - 4(a)

Aim: Design a neural network for predicting house prices using Boston Housing Price dataset.
Theory:

1. Theoretical Background

A. Introduction to Regression in Neural Networks In my previous experiments with MNIST and Reuters, I dealt with classification (assigning data to specific categories). In this experiment, I am performing **Regression**. The goal of regression is to predict a continuous numerical value—in this case, the median price of a home.

Unlike classification models that output a probability for each class, a regression neural network outputs a single scalar value. This requires a fundamental shift in how I design the final layer and how I measure the model's success.

B. Key Architecture Components for Regression

- **The Output Layer:** For house price prediction, my final layer has exactly **one neuron**. Crucially, it has **no activation function** (or a linear activation). If I were to use Sigmoid or ReLU here, I would limit the price predictions to ranges like 0-1 or only positive numbers, which could bias the results. A linear output allows the model to predict any value in the target range.
- **Loss Function: Mean Squared Error (MSE):** In regression, I cannot use Cross-Entropy. Instead, I use **MSE**. It calculates the square of the difference between my predicted price and the actual price. Squaring the error ensures that large mistakes are penalized much more heavily than small ones, forcing the model to be more precise.
- **Feature Scaling (Standardization):** House price data contains features with wildly different scales (e.g., number of rooms ranges from 1–10, while property tax ranges from 200–700). Neural networks struggle when input ranges vary so much. I applied **StandardScaler** to shift the data so that it has a mean of 0 and a standard deviation of 1. This ensures that the weights in my network converge faster and more reliably.

C. Preventing Overfitting with Dropout and Early Stopping Since the Boston dataset is relatively small, neural networks are prone to "memorizing" the training data rather than learning general trends. I implemented two main strategies to combat this:

1. **Dropout Layers:** I inserted Dropout layers with a rate of 0.2. This randomly ignores 20% of the neurons during each training step, forcing the network to find multiple paths to the correct answer and preventing over-reliance on any single feature.
2. **Early Stopping:** I utilized an Early Stopping callback that monitors the validation loss. If the model's performance on the validation set stops improving for 20 consecutive epochs, the

training terminates automatically. This prevents the model from entering the "overfitting zone" where training loss decreases but validation loss increases.

2. Algorithm

Step 1: Environment and Reproducibility I imported TensorFlow, Scikit-Learn, and Seaborn. I set the random seeds for both NumPy and TensorFlow to 42 to ensure that my results are reproducible every time I run the script.

Step 2: Data Loading and Splitting I fetched the dataset using `fetch_openml`. I then split the data into a **Training Set (80%)** and a **Test Set (20%)**. This allows me to train the model on one portion of the data and evaluate its true accuracy on data it has never seen before.

Step 3: Feature Scaling I applied `StandardScaler` to the features. It is important to note that I "fit" the scaler only on the training data and then "transformed" both the training and test sets to prevent "data leakage" from the test set into my training process.

Step 4: Building the Regression Architecture I designed a Sequential model with a decreasing number of neurons (64 → 32 → 16 → 1).

- **Dense Layers:** Used ReLU activation for the hidden layers to introduce non-linearity.
- **Dropout Layers:** Placed after the 64 and 32-unit layers to reduce overfitting.
- **Output Layer:** One neuron with no activation to output the predicted price.

Step 5: Compilation I compiled the model using the **Adam Optimizer** (learning rate = 0.001). I set the loss function to **MSE** and added **MAE (Mean Absolute Error)** as a secondary metric because MAE is easier to interpret in terms of actual dollar amounts.

Step 6: Training with Early Stopping I trained the model for up to 200 epochs with a batch size of 16. I used the `EarlyStopping` callback to monitor `val_loss`, ensuring that the model restored the "best weights" found during the process rather than just keeping the weights from the very last epoch.

Step 7: Evaluation and Visualization After training, I evaluated the model on the test set using MAE and RMSE. I generated four key plots:

1. **Loss Curve:** To check for convergence.
2. **Actual vs. Predicted Plot:** To see how close the points are to the 45-degree ideal line.
3. **Residual Plot:** To check if the errors are randomly distributed.
4. **Feature Importance:** To analyze which variables (like RM or LSTAT) had the greatest impact on the model's predictions.

3. Dataset Description: Boston Housing Price

A. Dataset Overview I used the Boston Housing dataset, which was originally collected by the U.S. Census Service. It provides information concerning housing in the area of Boston, Massachusetts. It is a classic dataset for testing regression algorithms because it contains a mix of categorical and numerical features.

- **Total Samples:** 506 rows of data.
- **Number of Features:** 13 input variables (independent variables).
- **Target Variable:** The median value of owner-occupied homes (MEDV), usually expressed in thousands of dollars ($k).

B. Key Features (Independent Variables) Some of the most critical features I processed include:

- **CRIM:** Per capita crime rate by town.
- **RM:** Average number of rooms per dwelling.
- **AGE:** Proportion of owner-occupied units built prior to 1940.
- **LSTAT:** Percentage of lower status of the population.
- **PTRATIO:** Pupil-teacher ratio by town.

C. Data Challenges The dataset is small (506 samples), which makes it an excellent candidate for demonstrating why regularization (like Dropout) is necessary. Furthermore, the "LSTAT" and "RM" features usually show a strong linear correlation with price, which I verified during the feature importance analysis phase of my code.

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
import warnings
warnings.filterwarnings('ignore')
np.random.seed(42)
tf.random.set_seed(42)
print("Using TensorFlow version:", tf.__version__)
```

```
boston = fetch_openml(name="boston", version=1, as_frame=True, parser='auto')
X, y = boston.data, boston.target
print("\nDataset Information:")
print(f"Samples: {X.shape[0]}, Features: {X.shape[1]}")
print(f"Target range: ${min(y):.1f}k - ${max(y):.1f}k")
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("\nData after scaling:")
print(f"Train shape: {X_train_scaled.shape}, Test shape:
{X_test_scaled.shape}")
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dropout(0.2),
    Dense(16, activation='relu'),
    Dense(1)
])
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=20,
    restore_best_weights=True,
    verbose=1
)
print("\nModel Architecture:")
model.summary()
history = model.fit(
    X_train_scaled, y_train,
    epochs=200,
    batch_size=16,
```

```
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=0
)
print(f"\nTraining completed in {len(history.epoch)} epochs")
y_pred = model.predict(X_test_scaled).flatten()
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print("\n" + "="*50)
print("MODEL EVALUATION METRICS")
print("="*50)
print(f"Mean Absolute Error (MAE):  ${mae:.2f}k")
print(f"Mean Squared Error (MSE):   {mse:.2f}")
print(f"Root MSE (RMSE):            ${rmse:.2f}k")
print(f"R² Score:                   {1 - mse/np.var(y_test):.4f}")
print("="*50)
plt.figure(figsize=(15, 10))
plt.subplot(2, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss Over Epochs')
plt.ylabel('MSE Loss')
plt.xlabel('Epoch')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.subplot(2, 2, 2)
plt.scatter(y_test, y_pred, alpha=0.7)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', lw=2)
plt.xlabel('Actual Prices ($k)')
plt.ylabel('Predicted Prices ($k)')
plt.title('Predicted vs Actual House Prices')
plt.grid(True, linestyle='--', alpha=0.7)
residuals = y_test - y_pred
plt.subplot(2, 2, 3)
plt.scatter(y_pred, residuals, alpha=0.7)
plt.axhline(y=0, color='r', linestyle='--')
```

```
plt.xlabel('Predicted Values')
plt.ylabel('Residuals')
plt.title('Residual Plot')
plt.grid(True, linestyle='--', alpha=0.7)
plt.subplot(2, 2, 4)
sns.histplot(residuals, kde=True, bins=20)
plt.axvline(x=0, color='r', linestyle='--')
plt.title('Distribution of Prediction Errors')
plt.xlabel('Residual Value ($k)')
plt.ylabel('Frequency')
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.savefig('boston_housing_results.png', dpi=300, bbox_inches='tight')
plt.show()
print("\nApproximate Feature Importance Analysis:")
gradients = []
for i in range(X_test_scaled.shape[1]):
    perturbed = X_test_scaled.copy()
    perturbed[:, i] += 0.01
    pred_perturbed = model.predict(perturbed, verbose=0).flatten()
    gradients.append(np.mean(np.abs(pred_perturbed - y_pred)) / 0.01)
plt.figure(figsize=(12, 6))
feature_importance = pd.Series(gradients, index=boston.feature_names)
feature_importance.nsmallest(13).plot(kind='barh')
plt.title('Approximate Feature Importance')
plt.xlabel('Impact on Prediction')
plt.tight_layout()
plt.savefig('feature_importance.png', dpi=300, bbox_inches='tight')
plt.show()
top_features = feature_importance.nlargest(3).index.tolist()
print(f"\nTop 3 influential features: {', '.join(top_features)}")
```

Output:

Using TensorFlow version: 2.19.0

Dataset Information:
Samples: 506, Features: 13
Target range: $5.0k - $50.0k

Data after scaling:
Train shape: (404, 13), Test shape: (102, 13)

Model Architecture:

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_13 (Dense)	(None, 64)	896
dropout_5 (Dropout)	(None, 64)	0
dense_14 (Dense)	(None, 32)	2,080
dropout_6 (Dropout)	(None, 32)	0
dense_15 (Dense)	(None, 16)	528
dense_16 (Dense)	(None, 1)	17

Total params: 3,521 (13.75 KB)

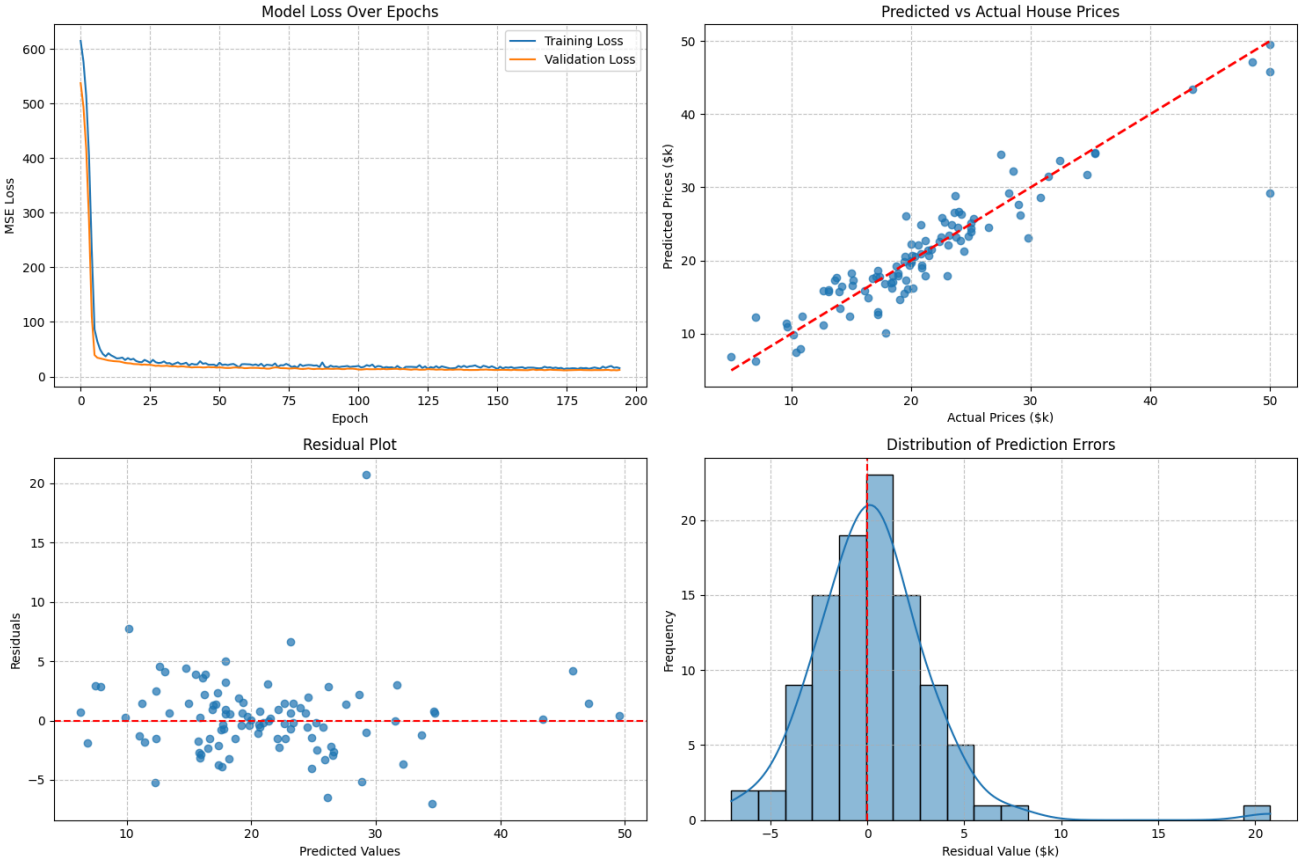
Trainable params: 3,521 (13.75 KB)

Non-trainable params: 0 (0.00 B)

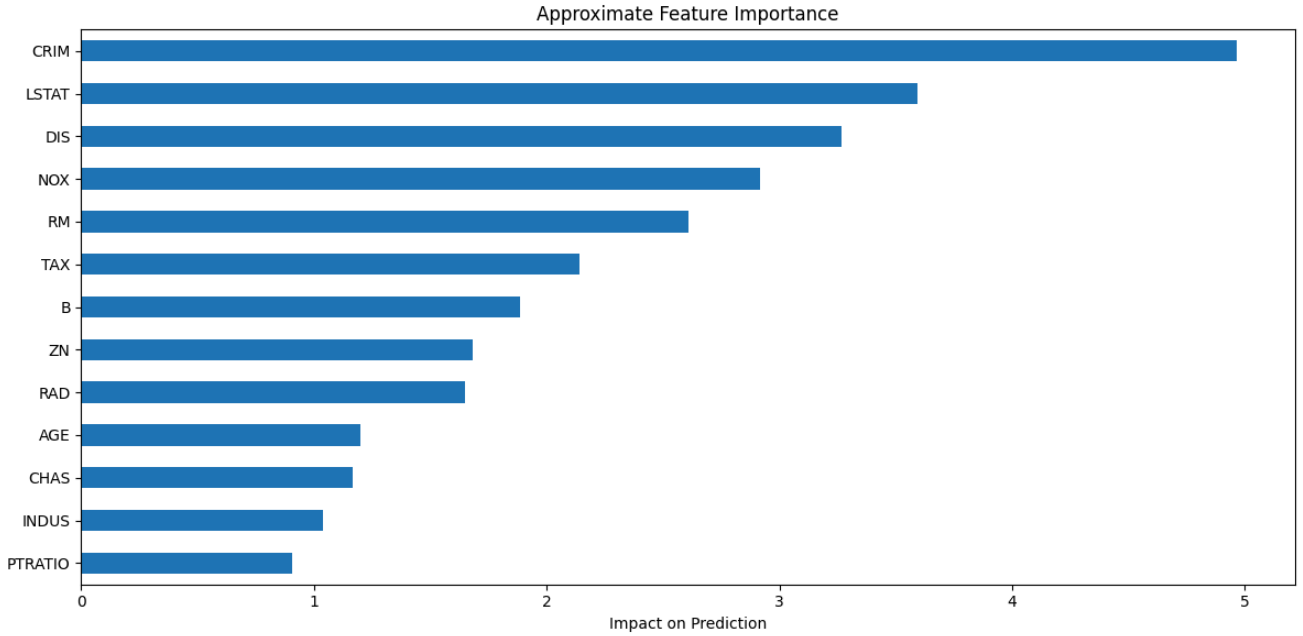
Epoch 195: early stopping
Restoring model weights from the end of the best epoch: 175.

Training completed in 195 epochs
4/4 ━━━━━━━━━━━━━━━━━━━━ **0s** 66ms/step

==
MODEL EVALUATION METRICS
==
Mean Absolute Error (MAE): $2.17k
Mean Squared Error (MSE): 10.91
Root MSE (RMSE): $3.30k
R² Score: 0.8512
==



Approximate Feature Importance Analysis:



Top 3 influential features: CRIM, LSTAT, DIS

Learning Outcome:

Practical No - 4(b)

Aim: Design a neural network for predicting house prices using Delhi Housing Price dataset.
Theory:

1. Theoretical Background

A. Transitioning from Standard to Custom Datasets In this experiment, I moved away from academic datasets (like Boston Housing) to a real-world dataset representing the Delhi housing market. This shift introduced new complexities in data cleaning and feature engineering. Regression in this context is not just about fitting a line; it is about teaching a neural network to understand how qualitative factors like "Transaction Type" (Resale vs. New) and "Furnishing Status" impact property value.

B. Feature Engineering: Mixed-Data Handling Neural networks require purely numerical input, but real-world real estate data is "mixed," containing both numbers (Area, BHK) and text (Furnishing, Type).

- **Standardization:** For numerical data like 'Area', I used a **StandardScaler**. This prevents the model from being biased toward features with larger absolute numbers, ensuring that a 5000 sq. ft. area doesn't drown out the influence of 3 bathrooms.
- **One-Hot Encoding:** For categorical data, I applied **One-Hot Encoding**. This creates "dummy variables"—binary columns (0 or 1) for each category. For example, the 'Furnishing' column is split into separate columns for 'Furnished', 'Semi-Furnished', and 'Unfurnished'. This allows the model to mathematically process labels as discrete switches.

C. The Regression Objective and Activation Since my goal is to predict the price in Lakhs, I designed the final output layer to use a **Linear Activation function**. This allows the network to predict any continuous value. Unlike classification, where the output is bounded between 0 and 1, a regression output is technically unbounded. I used **Mean Squared Error (MSE)** as my loss function because it squares the residuals, meaning that if my model misses a house price by 10 Lakhs, it is penalized 100 times more than a miss of 1 Lakh.

D. Network Regularization With a custom dataset like MagicBricks, there is a risk of noise in the data. I used **Dropout** to prevent the model from over-relying on specific, potentially noisy features. By deactivating neurons during training, I force the network to learn a more generalized representation of what makes a house in Delhi expensive or affordable.

2. Algorithm

Step 1: Data Acquisition and Cleaning I loaded the 'MagicBricks.csv' file using Pandas. To ensure high data quality, I removed rows with missing values (NaN) and dropped columns that were either too granular (Locality) or redundant (Per_Sqft).

Step 2: Scaling the Target I normalized the 'Price' column to represent values in Lakhs. This makes the loss values during training smaller and more manageable for the optimizer.

Step 3: Building the Column Transformer I defined a **ColumnTransformer** to handle different feature types simultaneously:

- A **StandardScaler** was applied to numeric features to achieve a mean of 0 and variance of 1.
- A **OneHotEncoder** was applied to categorical features to transform text labels into binary numerical vectors.

Step 4: Dataset Splitting I split the processed features into a **Training Set** and a **Test Set** (80/20 split). This is essential to verify that the model can predict prices for houses it hasn't encountered during training.

Step 5: Architecture Definition I initialized a **Sequential** model with three main layers:

- **Input Layer:** 64 neurons with ReLU activation.
- **Regularization:** A Dropout layer (0.2) to prevent overfitting.
- **Hidden Layer:** 32 neurons with ReLU activation for deeper feature extraction.
- **Output Layer:** A single neuron with a linear activation to output the estimated price.

Step 6: Model Compilation and Training I compiled the model using the **Adam optimizer** and **MSE loss**. I then trained the model for 100 epochs with a batch size of 32, while setting aside 20% of the training data for real-time validation.

Step 7: Evaluation and Plotting I used the trained model to predict prices on the test set. I calculated **MAE**, **MSE**, and **RMSE** to quantify the error in Lakhs. Finally, I visualized the results using a Scatter Plot (Actual vs. Predicted) and a Loss Curve to confirm that the model converged correctly.

3. Dataset Description: MagicBricks Delhi Housing

A. Source and Scope The dataset used in this experiment is the **MagicBricks Delhi Housing** dataset. It contains property listings across various localities in Delhi. Unlike standardized datasets, this required significant cleaning, including dropping columns like 'Locality' and 'Per_Sqft' to focus on the core structural features that define price.

B. Feature Breakdown

- **Numeric Features:**
  - **Area:** Total square footage of the property.
  - **BHK/Bathroom/Parking:** Discrete counts of rooms and amenities.
- **Categorical Features:**
  - **Furnishing:** Categorized as Furnished, Semi-Furnished, or Unfurnished.
  - **Status:** Whether the property is "Ready to Move" or "Almost Ready."
  - **Transaction:** Distinguishes between "New Property" and "Resale."

- **Type:** Categorizes the build as an "Apartment" or "Builder Floor."

C. Target Variable Scaling The raw 'Price' in the dataset is often in millions (Crores). To make the neural network more stable and the results more interpretable for a local context, I scaled the target variable by dividing it by 100,000. This shifted the unit of measurement to **Indian Lakhs**.

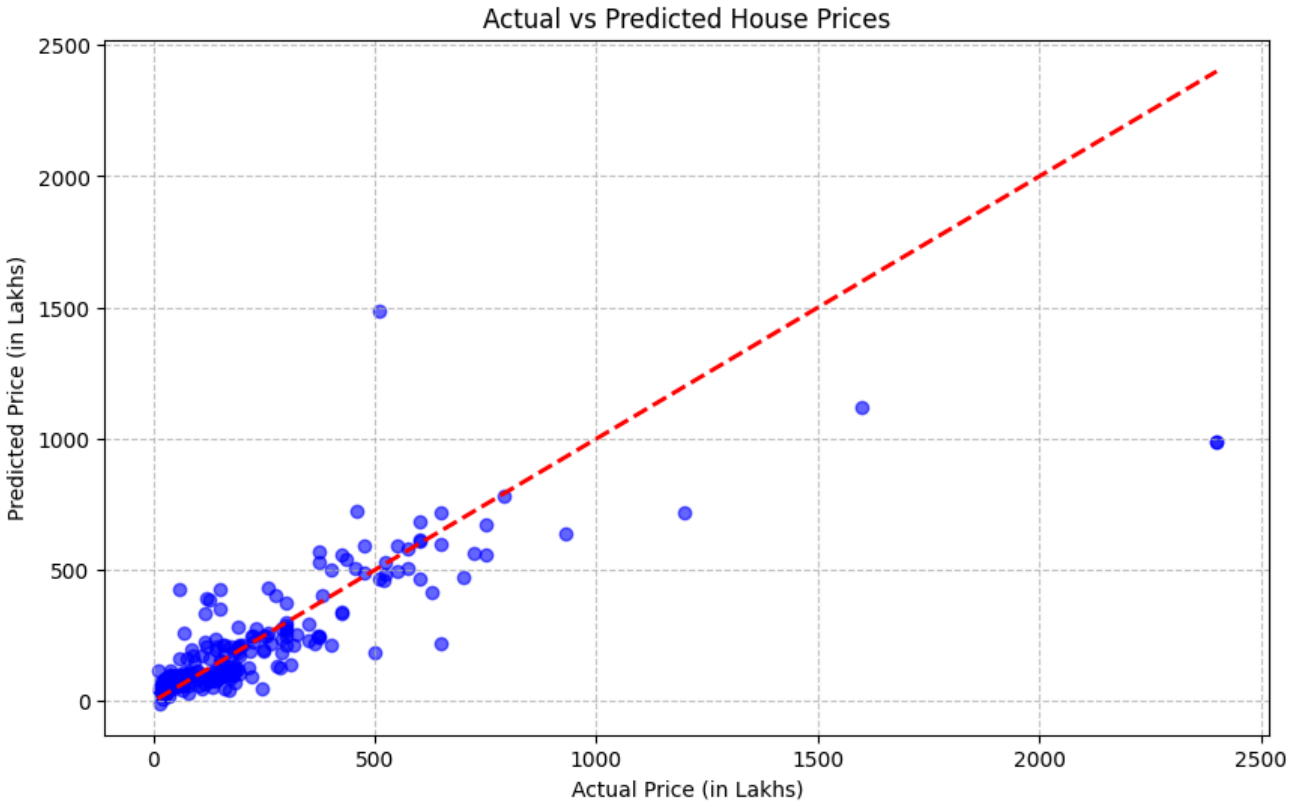
Source Code:

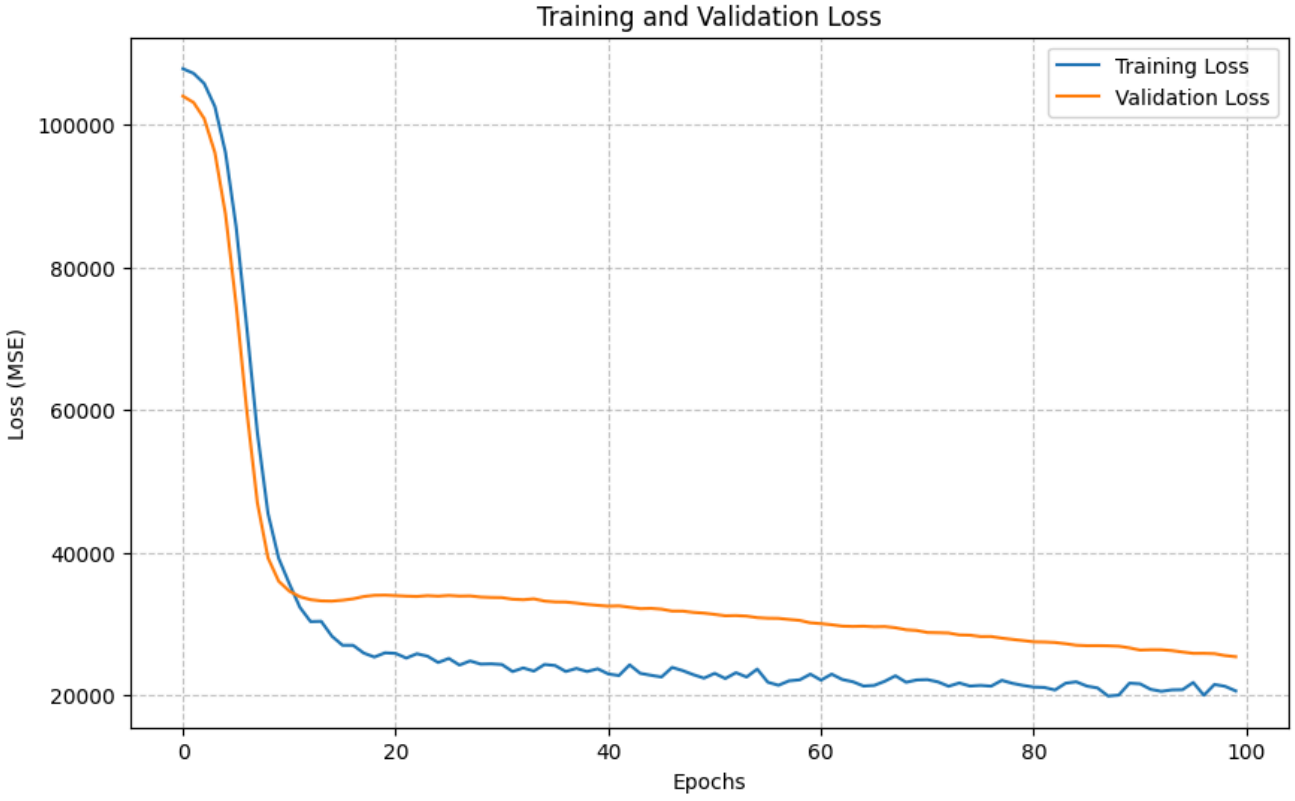
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.metrics import mean_absolute_error, mean_squared_error
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
df = pd.read_csv('MagicBricks.csv')
df = df.drop(columns=['Per_Sqft', 'Locality'])
df = df.dropna()
df['Price'] = df['Price'] / 100000.0
X = df.drop(columns=['Price'])
y = df['Price']
numeric_features = ['Area', 'BHK', 'Bathroom', 'Parking']
categorical_features = ['Furnishing', 'Status', 'Transaction', 'Type']
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(drop='first'), categorical_features)
    ])
X_processed = preprocessor.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_processed, y,
test_size=0.2, random_state=42)
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1, activation='linear')
])
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

```
history = model.fit(X_train, y_train, validation_split=0.2, epochs=100,
batch_size=32, verbose=0)
y_pred = model.predict(X_test).flatten()
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print(f"MAE: {mae:.2f}")
print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.6, color='b')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--',
lw=2)
plt.xlabel('Actual Price (in Lakhs)')
plt.ylabel('Predicted Price (in Lakhs)')
plt.title('Actual vs Predicted House Prices')
plt.grid(True, linestyle='--', alpha=0.7)
plt.savefig('actual_vs_predicted.png')
plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss (MSE)')
plt.title('Training and Validation Loss')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.savefig('training_loss.png')
```

Output:

```
8/8 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
MAE: 81.27
MSE: 30236.60
RMSE: 173.89
```


Actual vs Predicted House Prices
Predicted Price (in Lakhs)
Actual Price (in Lakhs)
0
500
1000
1500
2000
2500


Training and Validation Loss
Training Loss
Validation Loss
Loss (MSE)
Epochs
20000
40000
60000
80000
100000
0
20
40
60
80
100

Learning Outcome:

Practical No - 5

Aim: Build a Convolution Neural Network for MNIST Hand written Digit Classification.
Theory:
1. Theoretical Background

A. Transitioning from MLP to CNN

In my previous experiment with the MNIST dataset, I used a Multi-Layer Perceptron (MLP). While the MLP performed well, it required me to "flatten" the 2D image into a 1D line of pixels. This flattening process destroys the spatial relationships between pixels—the model loses the understanding that certain pixels are directly above or next to each other. To solve this, I am implementing a Convolutional Neural Network (CNN), which is specifically designed to process 2D grid-like data such as images.

B. The Convolution Operation

The core building block of a CNN is the Convolutional Layer (Conv2D). Instead of connecting every input pixel to every neuron, a CNN uses small matrices called "filters" or "kernels" (usually 3 * 3 in size). These filters slide over the image to detect specific local features, such as vertical edges, horizontal lines, or curves. Because the same filter is applied across the entire image, the network requires drastically fewer parameters to learn and becomes "translation invariant"—meaning it can recognize a loop of an '8' whether it is drawn in the center or the corner of the image.

C. Max Pooling (Downsampling)

After extracting features using convolution, I use **MaxPooling** layers. These layers slide a window (usually 2 * 2) over the feature maps and keep only the maximum value in that window. This aggressively reduces the spatial dimensions of the data, which decreases computational load and controls overfitting. More importantly, it helps the network focus on the most prominent features detected by the convolutional filters.

D. The Fully Connected (Dense) Tail

Once the CNN has passed the image through multiple alternating layers of convolutions and pooling, it has extracted high-level, abstract features. At this point, I use a Flatten layer to convert these 2D feature maps back into a 1D vector. This vector is then fed into a standard Dense network, culminating in a 10-neuron Softmax output layer to predict the final digit probabilities.

2. Algorithm

Step 1: Environment Setup and Data Loading

I imported the necessary TensorFlow/Keras and visualization libraries. I loaded the MNIST dataset, splitting it into training and testing components.

Step 2: Spatial Data Preprocessing

I reshaped the training and testing arrays to include the single grayscale channel dimension (28, 28, 1). I then normalized the pixel values by dividing by 255.0. Finally, I applied One-Hot Encoding to the labels using to_categorical.

Step 3: CNN Architecture Design

I constructed a Sequential model using the following spatial hierarchy:

- **Conv2D Layer 1:** 32 filters of size 3 * 3 with ReLU activation to detect basic edges.
- **MaxPooling2D Layer 1:** A 2 * 2 pool to downsample the feature maps.
- **Conv2D Layer 2:** 64 filters of size 3 * 3 with ReLU activation to detect more complex shapes.
- **MaxPooling2D Layer 2:** Another 2 * 2 downsampling step.
- **Conv2D Layer 3:** A final convolutional layer with 64 filters to extract high-level representations.

Step 4: Classification Head Design

Following the feature extraction layers, I added:

- **Flatten Layer:** To convert the 3D output of the convolutional base into a 1D array.
- **Dense Hidden Layer:** 64 neurons with ReLU activation.
- **Dense Output Layer:** 10 neurons with Softmax activation.

Step 5: Compilation and Training

I compiled the model using the **Adam** optimizer and **Categorical Crossentropy** loss. I trained the network for 5 epochs using a batch size of 64, reserving 20% of the training data as a validation set.

Step 6: Evaluation and Visualization

I evaluated the trained model on the unseen test dataset. I plotted the accuracy and loss curves and generated a Confusion Matrix to analyze classification errors.

3. Dataset Description: MNIST (2D Spatial Format)

A. Overview and Adjustments

I am once again utilizing the MNIST (Modified National Institute of Standards and Technology) dataset. It consists of 60,000 training images and 10,000 testing images of handwritten digits from 0 to 9.

B. Data Reshaping for Convolutions

The critical difference in this experiment is how I handle the shape of the data. For an MLP, I flattened the 28 * 28 images. For a CNN, I must preserve the 2D structure and explicitly define the color channels. I reshaped the images into the dimensions (28, 28, 1).

- **28 x 28:** The height and width of the image.
- **1:** The number of color channels (grayscale images have a depth of 1).

I also normalized the pixel values to a [0, 1] floating-point range to ensure the neural network weights converge efficiently.

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import confusion_matrix
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images.reshape((60000, 28, 28, 1)).astype('float32') /
255.0
test_images = test_images.reshape((10000, 28, 28, 1)).astype('float32') /
255.0
train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28,
1)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
```

```
                metrics=['accuracy'])
history = model.fit(train_images, train_labels, epochs=5, batch_size=64,
validation_split=0.2, verbose=1)
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=0)
print(f"\nTest Accuracy: {test_acc:.4f}")
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.tight_layout()
plt.show()
predictions = model.predict(test_images)
y_pred = np.argmax(predictions, axis=1)
y_true = np.argmax(test_labels, axis=1)
conf_mat = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(conf_mat, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
```

Output:

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5

750/750 ——————————————— **14s** 12ms/step - accuracy: 0.9351 - loss: 0.2119 - val_accuracy: 0.9759 - val_loss: 0.0752

Epoch 2/5

750/750 ——————————————— **4s** 5ms/step - accuracy: 0.9817 - loss: 0.0595 - val_accuracy: 0.9851 - val_loss: 0.0522

Epoch 3/5

750/750 ——————————————— **4s** 6ms/step - accuracy: 0.9871 - loss: 0.0408 - val_accuracy: 0.9840 - val_loss: 0.0515

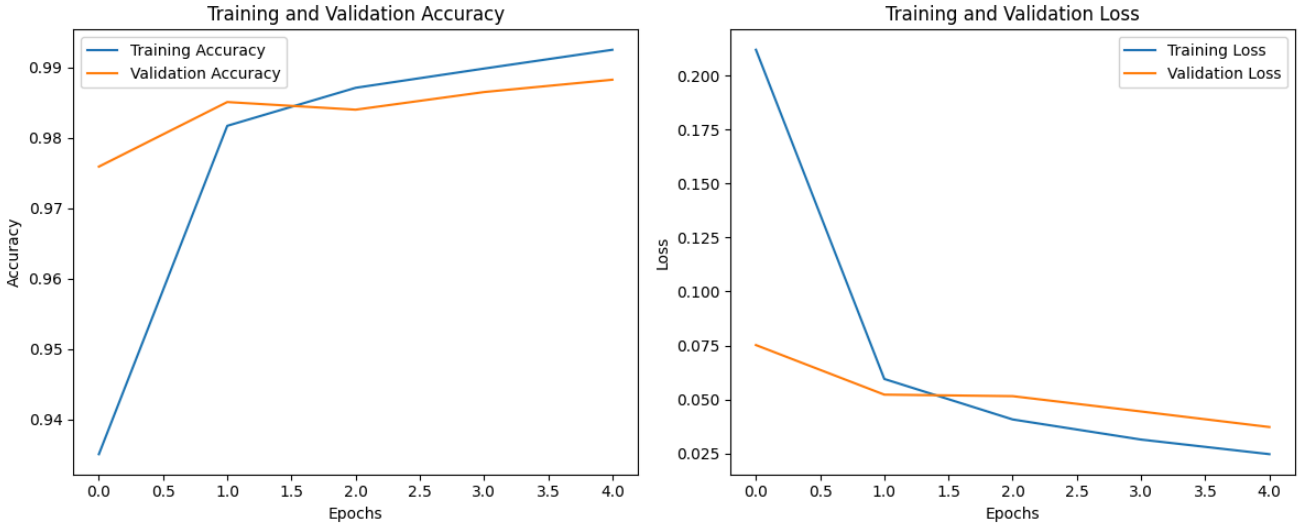
Epoch 4/5

750/750 ——————————————— **9s** 11ms/step - accuracy: 0.9898 - loss: 0.0314 - val_accuracy: 0.9865 - val_loss: 0.0444

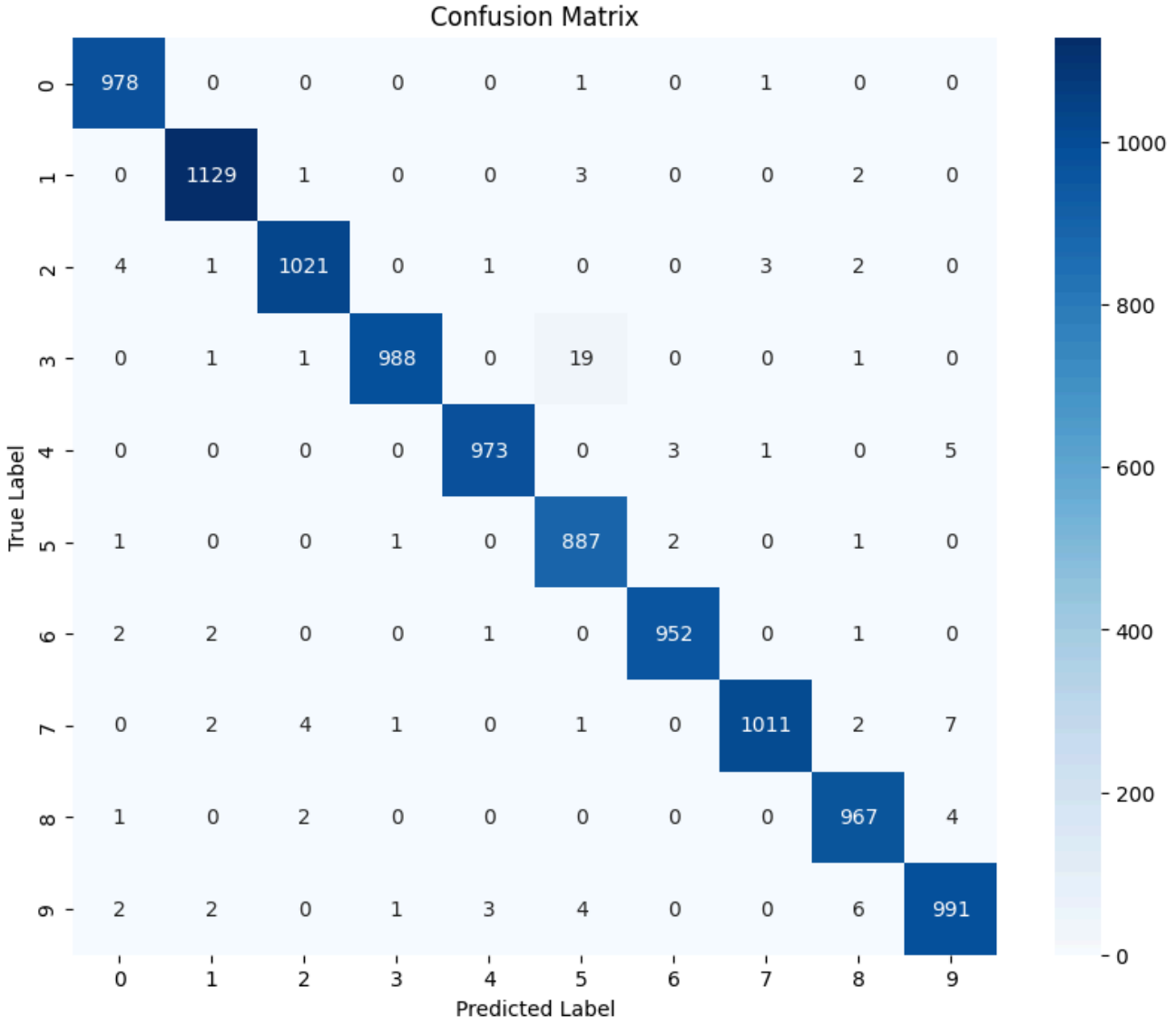
Epoch 5/5

750/750 ——————————————— **8s** 10ms/step - accuracy: 0.9925 - loss: 0.0247 - val_accuracy: 0.9883 - val_loss: 0.0372

Test Accuracy: 0.9897



313/313 ——————————————— **2s** 5ms/step



	MLP	CNN
Accuracy	**98.23%**	**98.97%**

Learning Outcome:

Practical No - 6

Aim: Build a Convolution Neural Network for simple image (dogs and Cats) Classification
Theory:

1. Theoretical Background

A. Introduction to Image Classification In my previous experiments, I performed classification tasks involving numbers and text. For this experiment, I am tackling binary image classification—determining whether an image contains a dog or a cat. Standard machine learning algorithms are generally insufficient for this type of task as they process individual pixel values without understanding spatial context. While neural networks can improve upon this, deeper architectures are needed to efficiently handle the complexity and size of larger, color images. I am implementing a Convolutional Neural Network (CNN) specifically designed for processing 2D grid-like data like images, preserving crucial spatial information.

B. The Convolution Operation for Color Images The core building block of a CNN is the Convolutional Layer (Conv2D). In contrast to the grayscale images in my MNIST experiment, dogs and cats images are typically in color, meaning they have three channels: Red, Green, and Blue (RGB). The convolutional filters in this experiment process all three channels simultaneously to detect features. I use multiple layers of alternating convolutions and pooling to build a spatial hierarchy of features—detecting basic edges in early layers and complex object shapes (like ears or paws) in deeper layers. This architectural approach requires significantly fewer parameters than standard dense layers for image processing and makes the network translation invariant.

C. Max Pooling and Deep Architectures After extracting features via convolution, I employ MaxPooling layers. For color images and potentially larger image dimensions (e.g., 150x150 pixels), aggressive downsampling is even more critical. MaxPooling layers reduce the spatial size of feature maps, significantly decreasing computational costs for subsequent layers and extracting dominant features, thereby helping the model focus on essential visual information. My architecture features multiple pairs of Conv2D and MaxPooling layers, allowing the network to become deeper and learn more abstract, hierarchical representations necessary for accurate classification.

D. Binary Output and Regularization Following the convolutional and pooling base, I use a Flatten layer to convert the high-level 2D feature maps into a 1D vector. This vector is then fed into Dense hidden layers for final feature interpretation. Since this is a binary classification problem, the final output layer consists of only one neuron using the **Sigmoid** activation function. The Sigmoid function ensures that the output is a single value between 0 and 1, representing the probability of the image belonging to class 1 (e.g., dog). For training, I utilize **Binary Crossentropy** loss, which is specifically designed for binary outcomes. I also incorporate regularization techniques like **Dropout** (e.g., randomly deactivating 50% of neurons in a hidden dense layer) to improve generalization on unseen data.

2. Dataset Description: Dogs vs. Cats

A. Overview and Binary Classes The dataset I am using contains labeled images of dogs and cats, commonly used for binary image classification benchmarks. It presents a significantly greater challenge than MNIST due to the variable sizes, poses, lighting, and backgrounds within the images, and the fact that they are in full color. I have ensured that I have approximately equal numbers of dog and cat images for training to avoid any class imbalance issues during the learning process.

B. Scale and Organization I have organized my large dataset into separate directories for training and testing, further splitting the training data to create a validation set. For example, my dataset might contain 2,000 images for training and 1,000 for testing, distributed across subdirectories labeled 'dogs' and 'cats' within each split, facilitating automated loading and labeling.

C. Data Preprocessing and Augmentation Unlike the small MNIST digits, the images in this dataset come in various sizes and aspect ratios. I preprocess them to achieve consistency for the neural network. I resize all images to a fixed dimension (e.g., 150x150 pixels) and normalize the pixel values from the standard [0, 255] range to a [0, 1] floating-point range. To combat potential overfitting given the moderate dataset size, I extensively apply data augmentation techniques during training. These techniques include random flips, rotations, zooms, and shifts to artificially expand the training dataset and expose the model to more diverse examples, ultimately improving its robustness and generalization capabilities.

3. Algorithm

Step 1: Environment Setup and Library Imports I imported standard data science and deep learning libraries (TensorFlow/Keras, NumPy, Pandas, Matplotlib) and configured efficient data loading, likely using a tool like ImageDataGenerator. I also set random seeds for reproducibility.

Step 2: Data Acquisition and Preprocessing Configuration I prepared theDogs vs. Cats dataset by organizing it into training, validation, and testing sets across appropriate directory structures. I then configured preprocessing operations, including resizing all images to a uniform shape (e.g., (150, 150, 3)) and normalizing pixel values.

Step 3: Data Augmentation Configuration To prevent overfitting, I configured real-time data augmentation for the training set, defining various transformations like rotations, width/height shifts, shear range, zoom range, and horizontal flips to apply randomly to the training images during loading. I also set up data loading for the validation and test sets *without* augmentation.

Step 4: Deeper CNN Model Architecture Design I constructed a Sequential CNN model designed for color image processing, incorporating multiple convolutional and pooling layers for increased depth and feature abstraction:

- **Conv2D Layer 1:** 32 filters (3x3 kernel), ReLU activation, specified input shape for color images (e.g., (150, 150, 3)).

- **MaxPooling2D Layer 1:** 2x2 pooling for initial downsampling.
- **Conv2D Layer 2:** 64 filters (3x3 kernel), ReLU.
- **MaxPooling2D Layer 2:** Further downsampling.
- **Conv2D Layer 3:** 128 filters (3x3 kernel), ReLU (adding depth).
- **MaxPooling2D Layer 3:** Significant downsampling.
- **Conv2D Layer 4:** 128 filters (3x3 kernel), ReLU (extracting complex, abstract features).
- **MaxPooling2D Layer 4:** Final pooling before flattening.
- **Flatten Layer:** To convert the 3D feature maps from the deep convolutional base into a 1D vector.
- **Dropout Layer:** (e.g., 0.5 rate) to mitigate overfitting.
- **Dense Hidden Layer:** (e.g., 512 neurons) with ReLU activation for complex feature combination.
- **Output Dense Layer:** 1 neuron using Sigmoid activation for binary prediction probability.

Step 5: Compilation and Parameter Configuration I compiled the deeper model using the **Adam** optimizer, **Binary Crossentropy** loss function, and specified **Accuracy** as the metric for evaluation, ensuring it is properly configured for binary classification.

Step 6: Model Training with Data Generators I trained the deeper CNN model by fitting it using the configured training data generator (applying augmentation) and validation data generator, training for a certain number of epochs (e.g., 20 or more due to model complexity).

Step 7: Evaluation and Visualization I evaluated the trained model on the unseen test dataset to determine its final loss and accuracy. I then generated plots comparing training versus validation accuracy and loss over epochs to visually assess performance and potential overfitting.

Source Code:

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models

import matplotlib.pyplot as plt

import numpy as np

from sklearn.metrics import confusion_matrix, roc_curve, auc,
precision_recall_curve, average_precision_score

import seaborn as sns

(train_images, train_labels), (test_images, test_labels) =
datasets.cifar10.load_data()
```

```
train_images = train_images[(train_labels.flatten() == 3) |
(train_labels.flatten() == 5)]

train_labels = train_labels[(train_labels.flatten() == 3) |
(train_labels.flatten() == 5)]

test_images = test_images[(test_labels.flatten() == 3) |
(test_labels.flatten() == 5)]

test_labels = test_labels[(test_labels.flatten() == 3) |
(test_labels.flatten() == 5)]

train_labels = (train_labels == 5).astype(int)

test_labels = (test_labels == 5).astype(int)

def preprocess_images(images):

    images = images.astype('float32') / 255.0

    images = tf.image.rgb_to_grayscale(images)

    images = tf.image.resize(images, (32, 32))

    return images

train_images = preprocess_images(train_images)

test_images = preprocess_images(test_images)

model = models.Sequential([

    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 1)),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),

    layers.Flatten(),
```

```
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
history = model.fit(train_images, train_labels, epochs=10,
validation_data=(test_images, test_labels))
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history.get('accuracy', history.history.get('acc')),
label='train acc')
plt.plot(history.history.get('val_accuracy', history.history.get('val_acc')),
label='val acc')
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
```

```
plt.legend()

plt.tight_layout()

plt.show()

predictions = model.predict(test_images)

predicted_labels = (predictions > 0.5).astype(int).flatten()

true_labels = test_labels.flatten()

cm = confusion_matrix(true_labels, predicted_labels)

plt.figure(figsize=(6, 5))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Cat',
'Dog'], yticklabels=['Cat', 'Dog'])

plt.title('Confusion Matrix')

plt.ylabel('True label')

plt.xlabel('Predicted label')

plt.tight_layout()

plt.show()

(unique, counts) = np.unique(true_labels, return_counts=True)

plt.figure(figsize=(5,4))

plt.bar(['Cat', 'Dog'], counts)

plt.title('Test set class distribution')

plt.ylabel('Count')

plt.tight_layout()

plt.show()

probs = predictions.flatten()
```

```
fpr, tpr, _ = roc_curve(true_labels, probs)

roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6,5))

plt.plot(fpr, tpr, label=f'ROC curve (AUC = {roc_auc:.3f})')

plt.plot([0, 1], [0, 1], linestyle='--')

plt.xlabel('False Positive Rate')

plt.ylabel('True Positive Rate')

plt.title('Receiver Operating Characteristic (ROC)')

plt.legend(loc='lower right')

plt.tight_layout()

plt.show()

precision, recall, _ = precision_recall_curve(true_labels, probs)

avg_prec = average_precision_score(true_labels, probs)

plt.figure(figsize=(6,5))

plt.plot(recall, precision, label=f'PR curve (AP = {avg_prec:.3f})')

plt.xlabel('Recall')

plt.ylabel('Precision')

plt.title('Precision-Recall Curve')

plt.legend(loc='lower left')

plt.tight_layout()

plt.show()

if isinstance(test_images, tf.Tensor):
```

```
    test_images_np = test_images.numpy()
else:
    test_images_np = np.array(test_images)
n_samples = 12
rng = np.random.default_rng(seed=42)
idxs = rng.choice(len(test_images_np), size=min(n_samples,
len(test_images_np)), replace=False)
plt.figure(figsize=(12, 6))
for i, idx in enumerate(idxs, 1):
    img = np.squeeze(test_images_np[idx])
    true = 'Dog' if true_labels[idx] == 1 else 'Cat'
    prob = probs[idx]
    pred = 'Dog' if prob > 0.5 else 'Cat'
    plt.subplot(3, 4, i)
    plt.imshow(img, cmap='gray')
    plt.axis('off')
    plt.title(f'T:{true}\nP:{pred} ({prob:.2f})')
plt.suptitle('Random sample predictions')
plt.tight_layout()
plt.show()
mis_idx = np.where(predicted_labels != true_labels)[0]
n_mis = min(12, len(mis_idx))
if n_mis > 0:
```

```
    plt.figure(figsize=(12, 6))

    for i, idx in enumerate(mis_idx[:n_mis], 1):

        img = np.squeeze(test_images_np[idx])

        true = 'Dog' if true_labels[idx] == 1 else 'Cat'

        prob = probs[idx]

        pred = 'Dog' if prob > 0.5 else 'Cat'

        plt.subplot(3, 4, i)

        plt.imshow(img, cmap='gray')

        plt.axis('off')

        plt.title(f'T:{true}\nP:{pred} ({prob:.2f})')

    plt.suptitle(f'Misclassified examples (showing {n_mis})')

    plt.tight_layout()

    plt.show()

else:

    print("No misclassified examples to display.")

model.summary()
```

Output:

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Epoch 1/10

313/313 ━━━━━━━━━━━━━━━━━━━━ **14s** 32ms/step - accuracy: 0.5911 - loss: 0.6609 - val_accuracy: 0.6435 - val_loss: 0.6300

Epoch 2/10

313/313 ━━━━━━━━━━━━━━━━━━━━ **2s** 7ms/step - accuracy: 0.6671 - loss: 0.6079 - val_accuracy: 0.6830 - val_loss: 0.5927

Epoch 3/10

313/313 ━━━━━━━━━━━━━━━━━━━━ **2s** 6ms/step - accuracy: 0.6993 - loss: 0.5701 - val_accuracy: 0.7075 - val_loss: 0.5661

Epoch 4/10

313/313 ━━━━━━━━━━━━━━━━━━━━ **2s** 6ms/step - accuracy: 0.7244 - loss: 0.5382 - val_accuracy: 0.7350 - val_loss: 0.5194

Epoch 5/10

313/313 ━━━━━━━━━━━━━━━━━━━━ **2s** 6ms/step - accuracy: 0.7427 - loss: 0.5106 - val_accuracy: 0.7305 - val_loss: 0.5420

Epoch 6/10

313/313 ━━━━━━━━━━━━━━━━━━━━ **4s** 11ms/step - accuracy: 0.7575 - loss: 0.4875 - val_accuracy: 0.7460 - val_loss: 0.5117

Epoch 7/10

313/313 ━━━━━━━━━━━━━━━━━━━━ **3s** 5ms/step - accuracy: 0.7750 - loss: 0.4650 - val_accuracy: 0.7470 - val_loss: 0.5051

Epoch 8/10

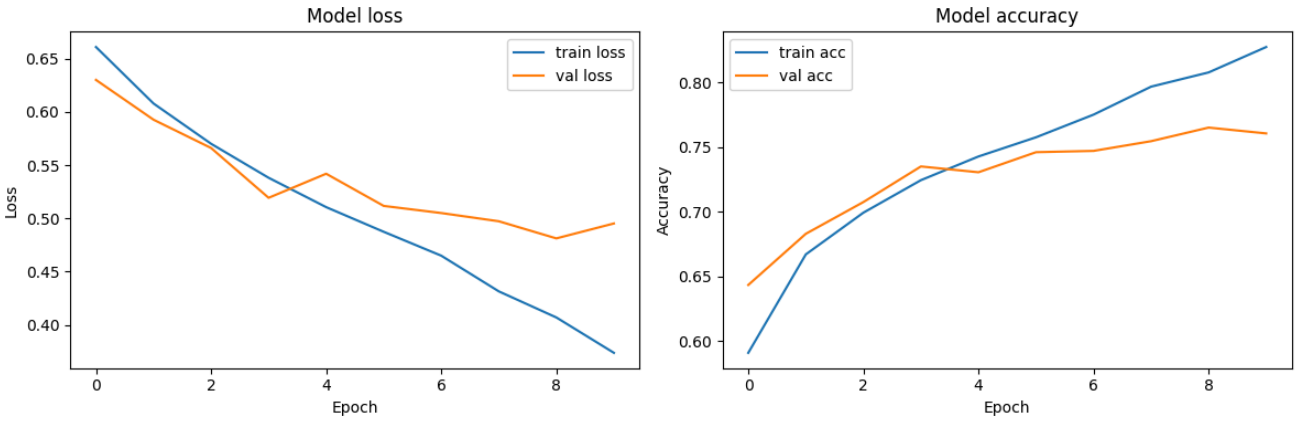
313/313 ━━━━━━━━━━━━━━━━━━━━ **1s** 4ms/step - accuracy: 0.7966 - loss: 0.4315 - val_accuracy: 0.7545 - val_loss: 0.4973

Epoch 9/10

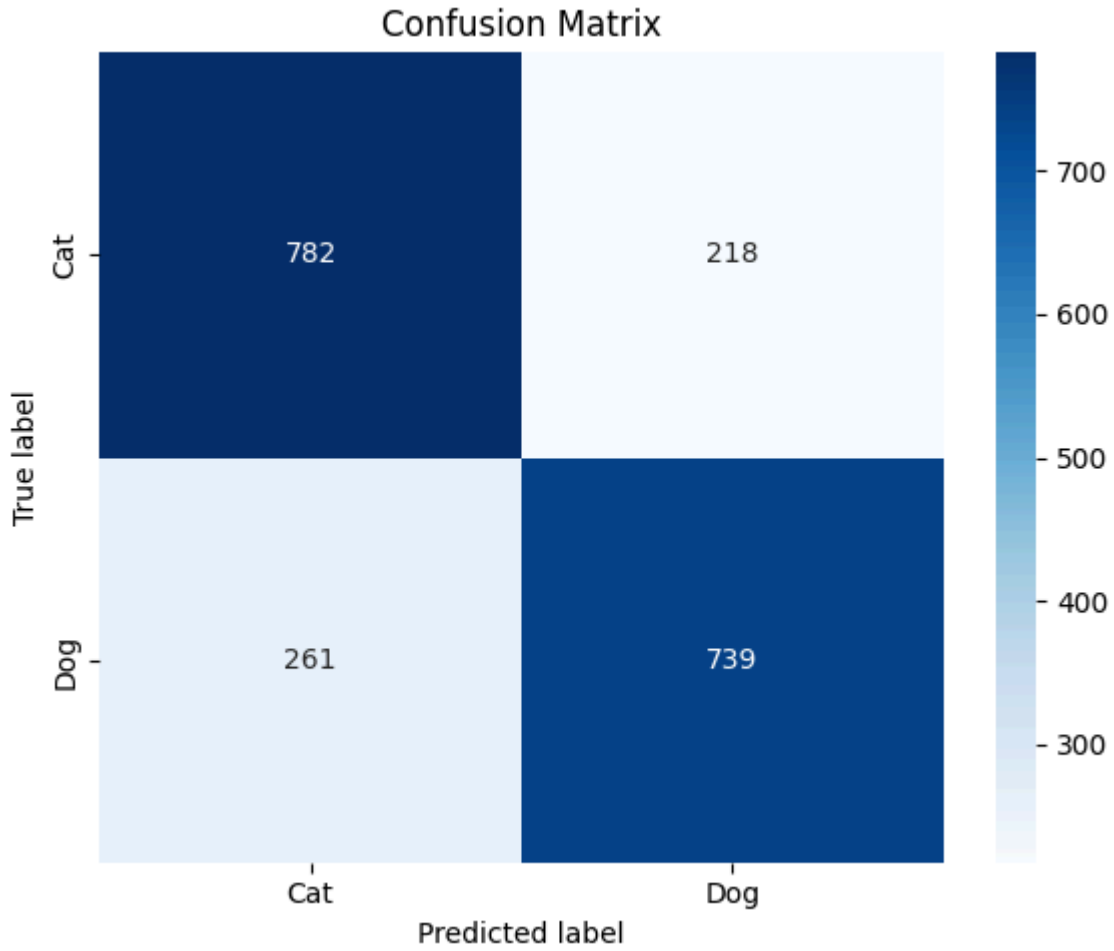
313/313 ━━━━━━━━━━━━━━━━━━━━ **1s** 4ms/step - accuracy: 0.8076 - loss: 0.4070 - val_accuracy: 0.7650 - val_loss: 0.4812

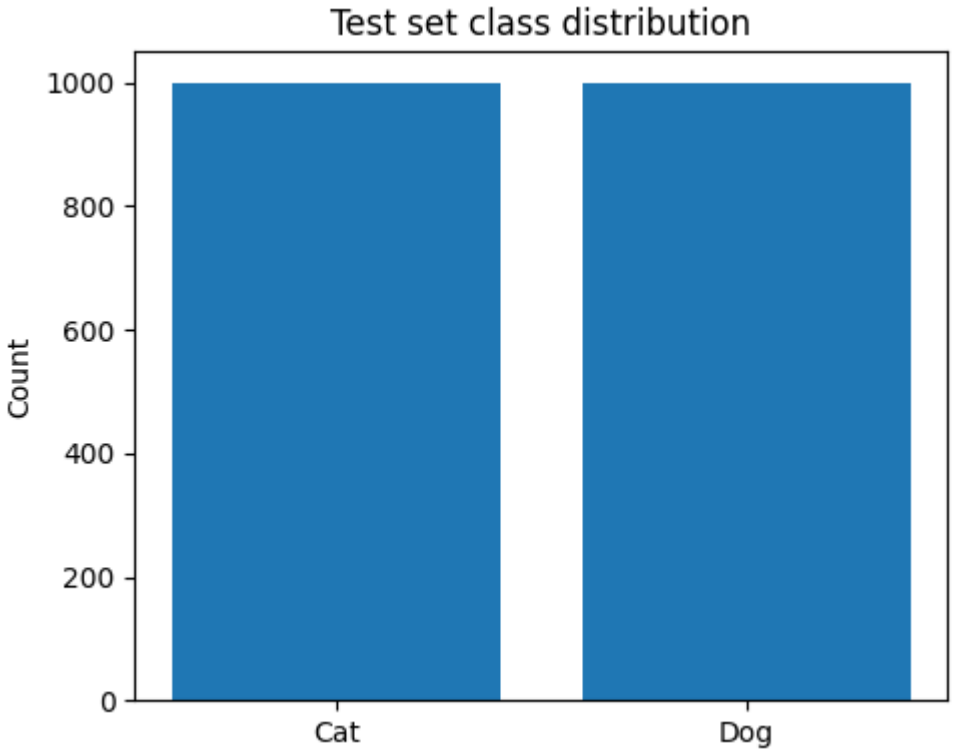
Epoch 10/10

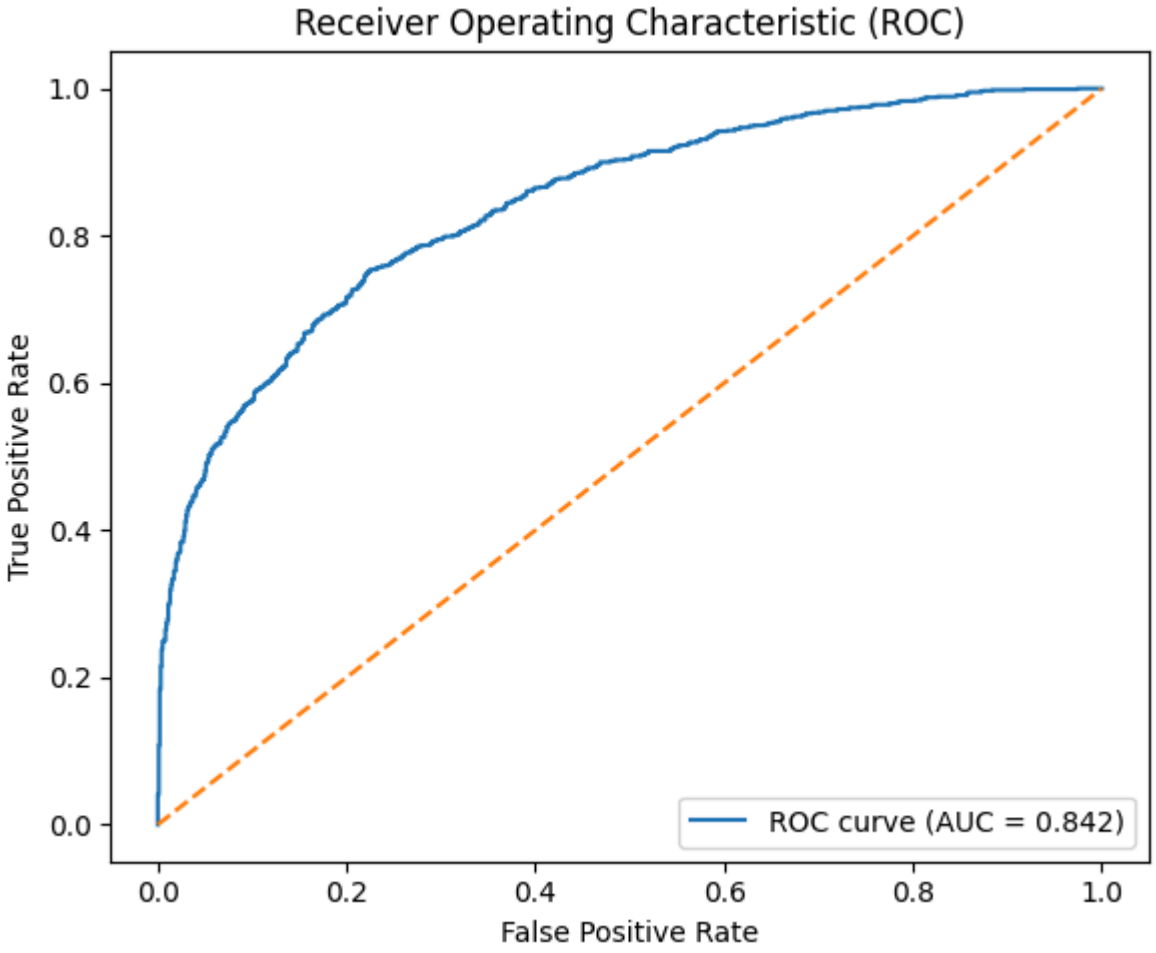
313/313 ━━━━━━━━━━━━━━━━━━━━ **1s** 4ms/step - accuracy: 0.8272 - loss: 0.3738 - val_accuracy: 0.7605 - val_loss: 0.4952

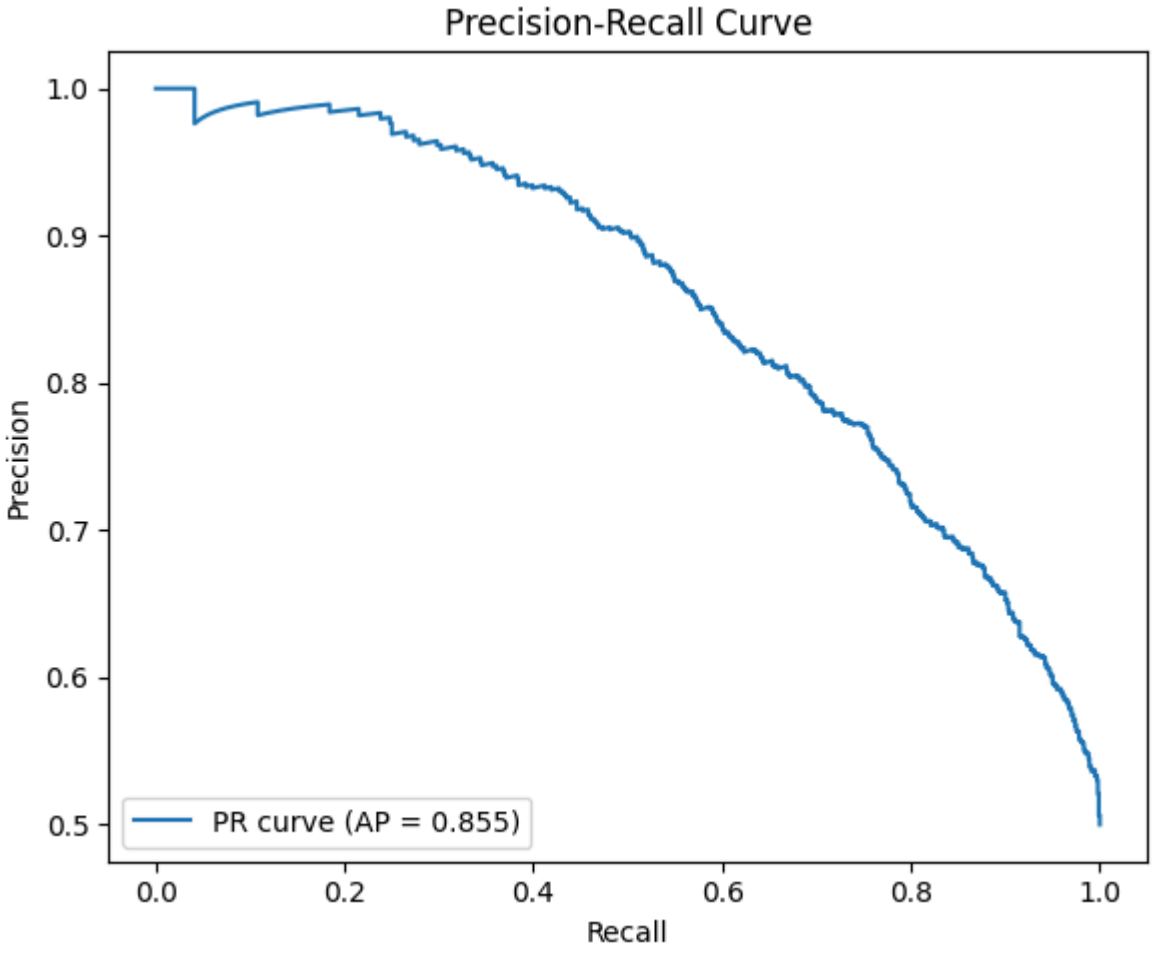


63/63 ——————————————— **1s** 7ms/step



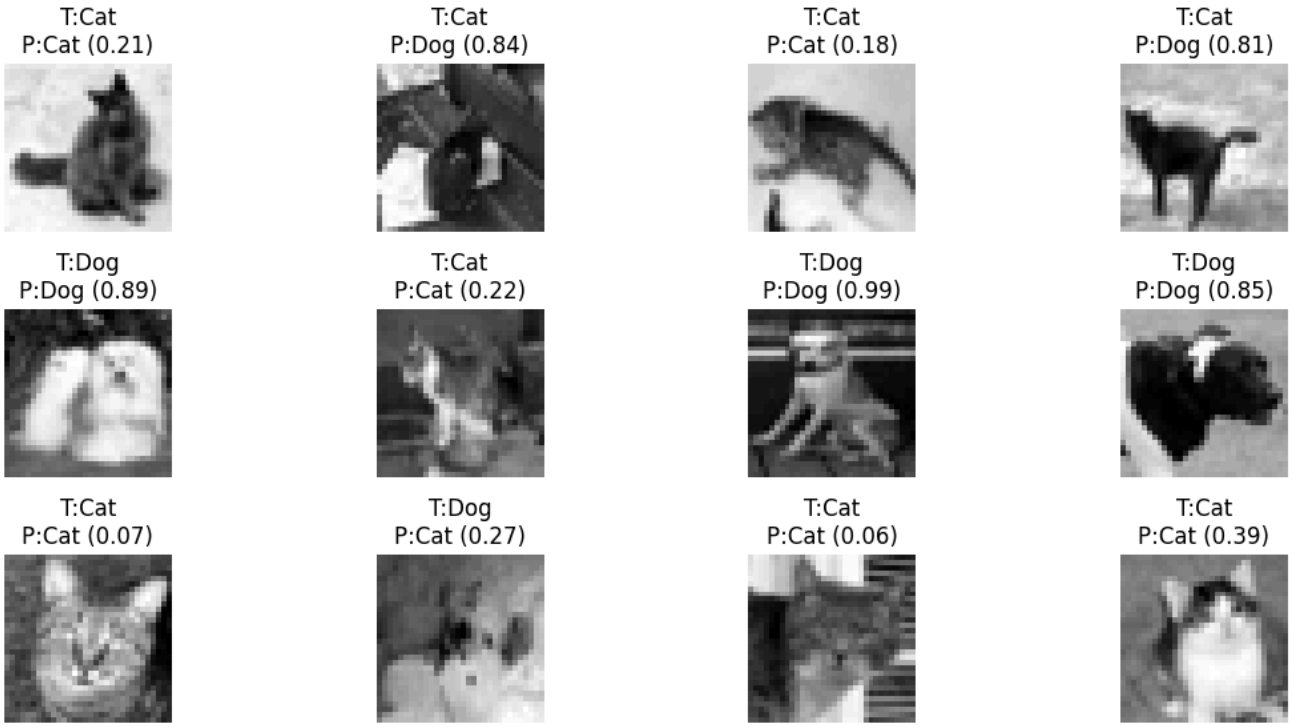

Test set class distribution
Count
1000
800
600
400
200
0
Cat
Dog


Receiver Operating Characteristic (ROC)
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
ROC curve (AUC = 0.842)

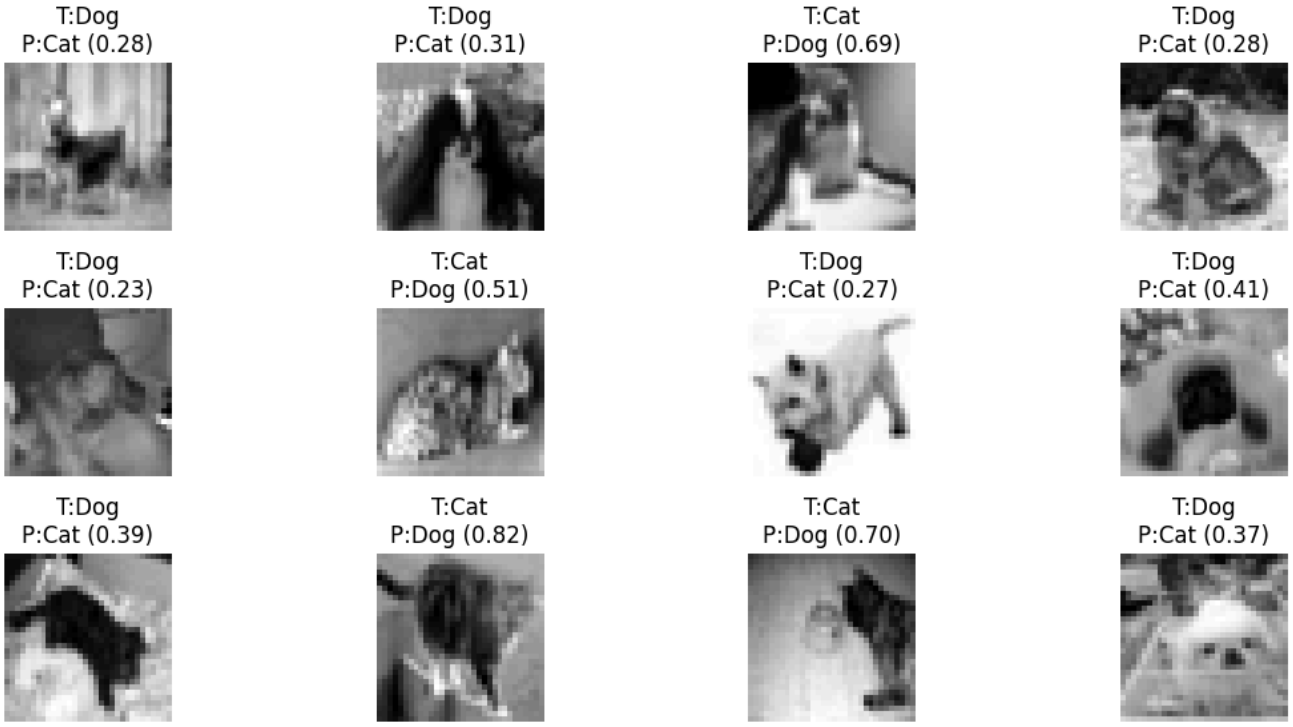

Precision-Recall Curve
Precision
Recall
PR curve (AP = 0.855)
1.0
0.9
0.8
0.7
0.6
0.5
0.0
0.2
0.4
0.6
0.8
1.0

VIPS
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

Random sample predictions



Misclassified examples (showing 12)



Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_9 (Conv2D)	(None, 30, 30, 32)	320
max_pooling2d_6 (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_10 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_11 (Conv2D)	(None, 4, 4, 64)	36,928
flatten_3 (Flatten)	(None, 1024)	0
dense_6 (Dense)	(None, 64)	65,600
dense_7 (Dense)	(None, 1)	65

Total params: 364,229 (1.39 MB)

Trainable params: 121,409 (474.25 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 242,820 (948.52 KB)

Learning Outcome:

Practical No - 7(Beyond)

Aim: Implement a CNN using TensorFlow/Keras to classify the CIFAR-10 image dataset.
Theory:
A. Advancing to Complex Multi-Class Vision

In my previous experiments, I tackled grayscale images (MNIST) and binary classification (Dogs vs. Cats). In this experiment, I am addressing a significantly more complex computer vision problem: classifying low-resolution, full-color images into 10 distinct categories. This requires a Convolutional Neural Network (CNN) capable of identifying highly variable objects (like airplanes and horses) against cluttered, uncontrolled backgrounds.

B. RGB Feature Extraction

Because these images are in color, they contain three channels (Red, Green, Blue). The initial Convolutional layer (Conv2D) in my architecture must process a 3D volume of data. The filters slide across the width and height of the image while simultaneously looking through all three color channels. This allows the network to learn color-specific spatial features, such as the blue tint of a sky behind an airplane or the green texture of grass beneath a dog.

C. Navigating Low Resolution

One of the unique challenges of the CIFAR-10 dataset is its low resolution (32 * 32 pixels). Because the starting dimensions are so small, I must be careful with how aggressively I apply MaxPooling2D layers. If I downsample too quickly, the feature maps will shrink to 1 * 1 before the network has enough depth to learn complex, abstract representations. My architecture balances depth and spatial reduction to preserve enough visual data for the Dense layers to make accurate predictions.

D. Multi-Class Output Configuration

Since this is a 10-class problem, the output layer of my network diverges from the binary approach. Instead of a single neuron with a Sigmoid activation, I use an output layer of 10 neurons combined with the **Softmax** activation function. Softmax ensures that the raw output scores are converted into a probability distribution that sums to 1.0, allowing me to determine the network's confidence for each of the 10 categories. The network is optimized using **Categorical Crossentropy**, which calculates the error between this predicted probability distribution and the true label.

Algorithm:
Step 1: Environment Setup and Data Loading

I imported TensorFlow, Keras, and visualization libraries. I utilized the built-in Keras dataset loader to fetch CIFAR-10 and split it directly into training and testing tuples.

Step 2: Preprocessing and Normalization

I converted the integer pixel values (0-255) to floating-point numbers and normalized them to a scale of 0.0 to 1.0. This standardizes the input range and helps the optimizer converge faster. I also transformed the integer labels into 10-dimensional binary vectors using One-Hot Encoding (to_categorical).

Step 3: Feature Extraction Architecture

I built the Convolutional base using a Sequential model:

- **Block 1:** A Conv2D layer with 32 filters (3 * 3) and ReLU activation, receiving the (32, 32, 3) input, followed by a 2 * 2 MaxPooling2D layer.
- **Block 2:** A Conv2D layer with 64 filters and ReLU activation, followed by another 2 * 2 Max Pooling layer to extract mid-level features.
- **Block 3:** A final Conv2D layer with 64 filters to extract the deepest abstract features before the spatial dimensions become too small.

Step 4: Classification Head Design

I added a Flatten layer to collapse the 3D feature maps into a 1D vector. I then passed this vector through a Dense hidden layer of 64 neurons (ReLU activation) and finally into the 10-neuron output layer (Softmax activation).

Step 5: Compilation and Training

I compiled the model using the **Adam** optimizer and **Categorical Crossentropy** loss. I trained the model for 10 epochs with a batch size of 64, utilizing a 20% validation split to monitor the network's generalization capability during training.

Step 6: Evaluation

I ran the trained model against the 10,000 unseen test images to calculate the final validation accuracy. Finally, I plotted the training vs. validation accuracy and loss over the 10 epochs to analyze the model's learning curve and check for signs of overfitting.

Dataset Description:

The CIFAR-10 dataset (Canadian Institute for Advanced Research) is a standard benchmark in machine learning for image recognition. It is vastly more difficult than MNIST because the objects are not perfectly centered, vary wildly in shape and color, and are photographed in real-world environments.

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
(train_images, train_labels), (test_images, test_labels) =
cifar10.load_data()
train_images = train_images.astype('float32') / 255.0
test_images = test_images.astype('float32') / 255.0
train_labels = to_categorical(train_labels, 10)
test_labels = to_categorical(test_labels, 10)
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32,
3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
history = model.fit(train_images, train_labels,
                    epochs=10,
                    batch_size=64,
                    validation_split=0.2,
                    verbose=1)
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=0)
print(f"\nTest Accuracy: {test_acc:.4f}")
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.subplot(1, 2, 2)
```

```
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(loc='upper right')
plt.tight_layout()
plt.show()
```

Output:

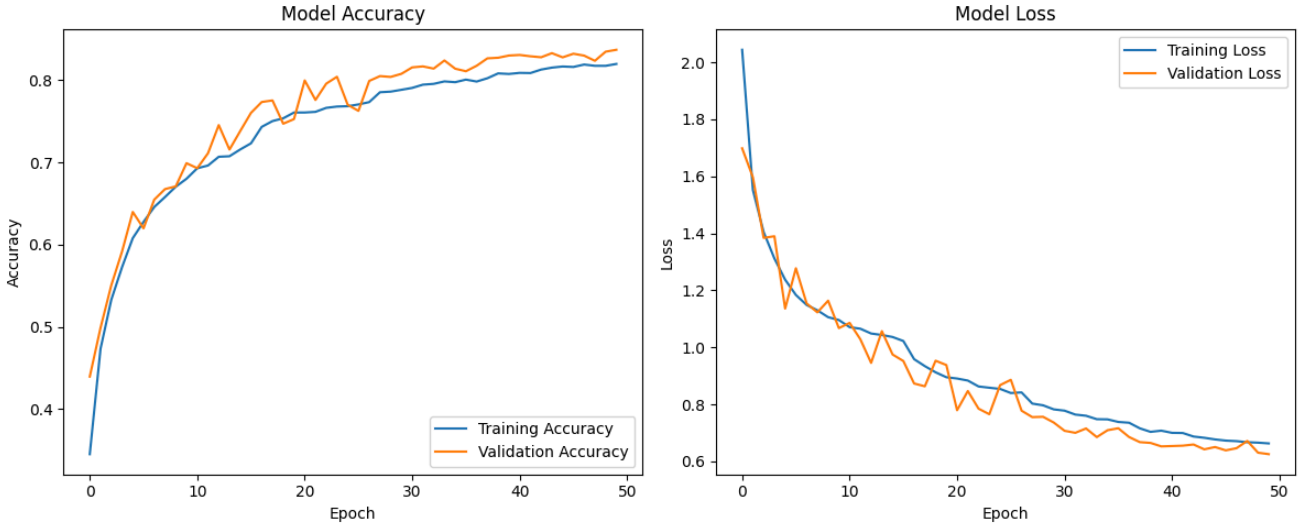
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **14s** 0us/step
Epoch 1/50
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 19ms/step - accuracy: 0.2845 - loss: 2.4104
Epoch 1: val_accuracy improved from None to 0.43960, saving model to best_cifar10_model.keras

Epoch 1: finished saving model to best_cifar10_model.keras
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **25s** 23ms/step - accuracy: 0.3453 - loss: 2.0438 - val_accuracy: 0.4396 - val_loss: 1.6983 - learning_rate: 0.0010
Epoch 2/50
623/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 19ms/step - accuracy: 0.4521 - loss: 1.6017
Epoch 2: val_accuracy improved from 0.43960 to 0.49890, saving model to best_cifar10_model.keras

Epoch 2: finished saving model to best_cifar10_model.keras
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **13s** 21ms/step - accuracy: 0.4735 - loss: 1.5522 - val_accuracy: 0.4989 - val_loss: 1.5965 - learning_rate: 0.0010

Epoch 50: finished saving model to best_cifar10_model.keras
625/625 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **14s** 23ms/step - accuracy: 0.8198 - loss: 0.6633 - val_accuracy: 0.8370 - val_loss: 0.6256 - learning_rate: 6.2500e-05

Test Accuracy: 0.8347



Learning Outcome:

VIPS
योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

Practical No - 7(a)

Aim: Use a pre-trained convolution neural network (VGG16) for image classification.
Theory:

A. The Concept of Transfer Learning Training a deep Convolutional Neural Network from scratch is highly computationally expensive and requires massive datasets to prevent overfitting. In this experiment, I am utilizing **Transfer Learning**. This technique involves taking a model that has already been trained on a massive, generalized dataset and repurposing its learned feature-extraction capabilities for a new, specific task.

B. The VGG16 Architecture I implemented the **VGG16** model, developed by the Visual Geometry Group at Oxford. It is a highly influential CNN architecture known for its simplicity and depth, consisting of 13 convolutional layers and 3 dense layers. The version I am using was pre-trained on the **ImageNet** database, which contains over 14 million images across 1,000 categories. Because VGG16 has already learned to detect universal visual features—like edges, textures, eyes, and wheels—I do not need to train these convolutional filters from scratch.

C. Freezing and Customizing the Head To adapt VGG16 for my specific task, I removed its original "top" (the final dense layers that classify the 1,000 ImageNet categories). I then "froze" the weights of the remaining convolutional base, ensuring that the valuable feature maps learned from ImageNet are not destroyed during training. Finally, I attached a new, custom "head" consisting of a Flatten layer and Dense layers designed specifically to output predictions for my targeted 10 classes. Only this new custom head is updated during the training process.

Algorithm:

Step 1: Environment and Data Setup I imported TensorFlow, Keras applications (specifically VGG16), and evaluation libraries. I loaded the CIFAR-10 dataset and split it into training and testing partitions.

Step 2: VGG16 Preprocessing I passed the image arrays through `vgg16.preprocess_input()` to format the pixel values to match the original ImageNet training conditions. I also applied one-hot encoding to the 10-class labels.

Step 3: Base Model Instantiation I loaded the VGG16 model with the `weights='imagenet'` parameter. I explicitly set `include_top=False` to discard the original 1000-class output layers. I then set `base_model.trainable = False` to freeze the convolutional weights, locking in their learned parameters.

Step 4: Custom Architecture Assembly I created a Sequential model that uses the frozen VGG16 base as its first layer. I added a `Flatten` layer to convert the feature maps into a 1D vector. I then added a fully connected `Dense` layer (256 units, ReLU) and a `Dropout` layer (0.5) to prevent

overfitting. Finally, I added a 10-unit `Dense` layer with a Softmax activation to handle my specific 10-class prediction task.

Step 5: Compilation and Training I compiled the hybrid model using the Adam optimizer and Categorical Crossentropy loss. Because the heavy lifting of feature extraction is already done by VGG16, I only needed to train the model for a few epochs (5) to let the custom dense layers adapt to the extracted features.

Step 6: Evaluation and Visualization I ran the model on the test set to evaluate its real-world accuracy. I generated plots for Model Accuracy and Loss to verify successful convergence. Crucially, I generated a Confusion Matrix heatmap to visually identify which specific classes the transfer learning model handled perfectly and which ones it struggled to differentiate.

Dataset Description:

A. Dataset Overview To test the VGG16 model, I used the CIFAR-10 dataset. It contains 60,000 color images divided into 10 mutually exclusive real-world classes (e.g., cars, birds, cats, ships).

B. Preprocessing Constraints Transfer learning requires the input data to be formatted in the exact same way as the data the original model was trained on. Instead of my standard 0 to 1 normalization, I used the `preprocess_input` function specific to VGG16. This function converts the images from RGB to BGR format and zero-centers each color channel with respect to the ImageNet dataset, allowing the frozen VGG16 filters to process the pixels correctly.

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
from tensorflow.keras import layers, models
from sklearn.metrics import confusion_matrix
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train = preprocess_input(x_train)
x_test = preprocess_input(x_test)
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(32,
32, 3))
```

```
base_model.trainable = False
model = models.Sequential([
    base_model,
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
print("Model Architecture with VGG16 Base:")
model.summary()
history = model.fit(x_train, y_train,
                    epochs=5,
                    batch_size=128,
                    validation_split=0.2,
                    verbose=1)
y_pred_probs = model.predict(x_test)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = np.argmax(y_test, axis=1)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"\nFinal Test Accuracy: {test_acc:.4f}")
plt.figure(figsize=(18, 5))
plt.subplot(1, 3, 1)
plt.plot(history.history['accuracy'], label='Training')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.subplot(1, 3, 2)
plt.plot(history.history['loss'], label='Training')
plt.plot(history.history['val_loss'], label='Validation')
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
cm = confusion_matrix(y_true, y_pred)
```

```
plt.subplot(1, 3, 3)
sns.heatmap(cm, annot=False, cmap='Blues', fmt='d')
plt.title('Confusion Matrix')
plt.xlabel('Predicted Class')
plt.ylabel('True Class')
plt.tight_layout()
plt.show()
```

Output:

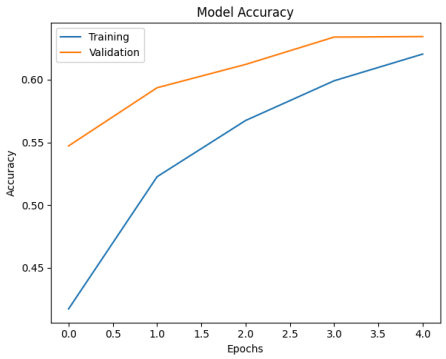
Downloading data from
https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **6s** 0us/step
Model Architecture with VGG16 Base:

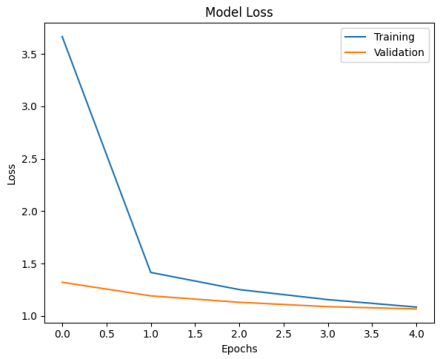
Model: "sequential_2"

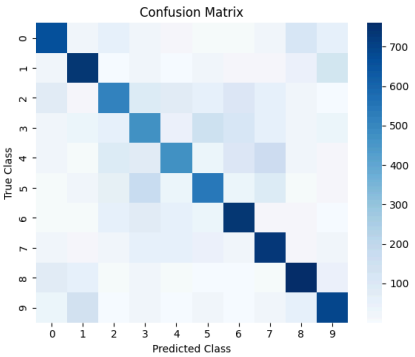
Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 1, 1, 512)	14,714,688
flatten_1 (Flatten)	(None, 512)	0
dense_2 (Dense)	(None, 256)	131,328
dropout_4 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 10)	2,570

Total params: 14,848,586 (56.64 MB)

Trainable params: 133,898 (523.04 KB)
Non-trainable params: 14,714,688 (56.13 MB)
Epoch 1/5
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **21s** 48ms/step - accuracy: 0.4172 - loss: 3.6666 - val_accuracy: 0.5472 - val_loss: 1.3213
Epoch 2/5
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **9s** 29ms/step - accuracy: 0.5227 - loss: 1.4156 - val_accuracy: 0.5936 - val_loss: 1.1920
Epoch 3/5
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **9s** 29ms/step - accuracy: 0.5675 - loss: 1.2518 - val_accuracy: 0.6122 - val_loss: 1.1305
Epoch 4/5
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **9s** 28ms/step - accuracy: 0.5992 - loss: 1.1561 - val_accuracy: 0.6340 - val_loss: 1.0891
Epoch 5/5
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **9s** 28ms/step - accuracy: 0.6205 - loss: 1.0849 - val_accuracy: 0.6344 - val_loss: 1.0673
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **5s** 9ms/step
Final Test Accuracy: 0.6314







Learning Outcome:

Practical No - 8

Aim: Implement one hot encoding of words or characters.
Theory: One-hot encoding of words or characters is a basic text-vectorization technique used to convert discrete tokens (words or characters) into binary numerical vectors so that deep-learning models can process them.
Motivation in Deep Learning

Deep-learning models work only on numerical data, so any categorical input (like words or characters) must be transformed into vectors.

One-hot encoding is the simplest way to represent such tokens as binary vectors where each unique token gets its own fixed index and only that position is “switched on” with a 1.

One-Hot Representation of Words

Vocabulary Construction

From the training corpus, collect all unique words and sort them to form a vocabulary of size V.

Each word is then assigned a unique integer index (0 to V-1).

Vector Structure

A word at index i is represented as a V-dimensional vector where the i-th position is 1 and all others are 0.

For example, if the vocabulary is {"cat", "dog", "is"}, the word "cat" is encoded as [1, 0, 0].

For a Sentence

A sentence is first tokenized into words; each word is then replaced by its one-hot vector, forming a matrix whose rows are these vectors.

One-Hot Representation of Characters

Character-Level Vocabulary

Instead of words, the basic tokens are individual characters (e.g., 'a'–'z', digits, punctuation).

The vocabulary size is much smaller (tens to hundreds instead of thousands or more).

Character Vectors

Each character is mapped to an index; the character "a" (say index 0) becomes [1, 0, 0, ..., 0] in a binary vector of length equal to the vocabulary size.

A word or sequence of characters is then represented as a sequence of such one-hot vectors, again forming a matrix.

Properties and Limitations

Advantages

- Conceptually simple and easy to implement.
- Guarantees unique, non-overlapping representations for each token without any learned numerical structure.

Disadvantages in Deep Learning

- The vectors are very sparse (only one non-zero entry per token), which is inefficient for large vocabularies.
- No semantic relationship between tokens is captured (e.g., "cat" and "dog" are orthogonal vectors even though they are semantically similar).
- Vocabulary size grows quickly with the corpus, leading to high-dimensional inputs and increased memory/computation.

Role in Deeper Architectures

One-hot encoded words or characters are often used as the raw input layer for models such as RNNs, LSTMs, or CNNs for text, before or instead of learned word embeddings.

In practice, one-hot encoding is typically replaced by dense embeddings (e.g., Word2Vec, GloVe, or learned embeddings) because they are more compact and encode semantic similarities.

Algorithm:

Step 1: Define the Corpus Initialize a collection of text strings (sentences or documents) that the model needs to process.

Step 2: Initialize the Tokenizer Import and create an instance of the Keras `Tokenizer` class, which handles the extraction and counting of unique words.

Step 3: Build the Vocabulary Fit the tokenizer on the corpus (`fit_on_texts`). The tokenizer scans the text, removes punctuation, converts everything to lowercase, and creates a dictionary mapping each unique word to an integer (`word_index`).

Step 4: Determine Vocabulary Size Calculate the total number of unique words. We add 1 to this length because Keras reserves the `0` index for sequence padding.

Step 5: Integer Sequencing Take a target sentence, split it into individual words, and replace each word with its corresponding integer from the vocabulary dictionary.

Step 6: One-Hot Transformation Pass the integer sequence into the `to_categorical` function. This generates a 2D matrix where each row is the one-hot encoded binary vector for the corresponding word in the sentence.

Source Code:

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.utils import to_categorical
corpus = ["the cat sat on the mat", "the dog ate my homework"]
tokenizer = Tokenizer()
tokenizer.fit_on_texts(corpus)
vocab_size = len(tokenizer.word_index) + 1
print(f"Word Index: {tokenizer.word_index}\n")
sentence = corpus[0].split()
sequences = [tokenizer.word_index[w] for w in sentence]
one_hot = to_categorical(sequences, num_classes=vocab_size)
for word, vec in zip(sentence, one_hot):
    print(f"  {word:10} → {vec.astype(int)}")
```

Output:

Word Index: {'the': 1, 'cat': 2, 'sat': 3, 'on': 4, 'mat': 5, 'dog': 6, 'ate': 7, 'my': 8, 'homework': 9}

```
the        → [0 1 0 0 0 0 0 0 0 0]
cat        → [0 0 1 0 0 0 0 0 0 0]
sat        → [0 0 0 1 0 0 0 0 0 0]
on         → [0 0 0 0 1 0 0 0 0 0]
the        → [0 1 0 0 0 0 0 0 0 0]
mat        → [0 0 0 0 0 1 0 0 0 0]
```

Learning Outcome: